

Máster Universitario en Data Analytics for Business

Trabajo Final de Máster

UPF Barcelona School of Management

Gestión del conocimiento organizacional mediante inteligencia
artificial generativa: diseño de una solución basada en
arquitectura RAG



Grupo número: 12

Juan Ignacio Botindari Rugnone

Marcel Portaz Blay

Isabella Rebolledo Domínguez

Tutora: Alexandra Abós Ortega

Junio 2026



Máster Universitario en Data Analytics for Business

Trabajo Final de Máster

UPF Barcelona School of Management

Gestión del conocimiento organizacional mediante inteligencia
artificial generativa: diseño de una solución basada en
arquitectura RAG

Grupo número: 12

Juan Ignacio Botindari Rugnone

Marcel Portaz Blay

Isabella Rebolledo Domínguez

Tutora: Alexandra Abós Ortega

Junio 2026

Tabla de contenidos

1.	Introducción.....	1
2.	Propuesta.....	2
3.	Resultados.....	3
3.1.	Los números de los titulares de segundo nivel no deben tener sangría.....	3
4.	Discusión.....	4
5.	Impacto Social.....	5
6.	Lista de figuras.....	6
7.	Lista de tablas.....	7
8.	Lista de símbolos.....	8
9.	Bibliografía.....	9
10.	Apéndices.....	10
10.1.	Apéndice 1: Dashboard.....	10
10.2.	Apéndice 2: Código y datos.....	10

Dedicación

(Opcional, si se usa, debe ir en una página derecha junto a una página izquierda vacía)

Reconocimientos

(Opcional, si se usa, debe ir en una página derecha junto a una página izquierda vacía)

Resumen

El presente Trabajo Fin de Máster (TFM) expone el diseño, desarrollo e implementación de una plataforma de software como servicio (SaaS) multi-tenant basada en una arquitectura de Generación Aumentada por Recuperación (RAG) multimodal. El núcleo de la investigación aborda la optimización en la gestión del conocimiento organizacional y la resolución de ineficiencias operativas en entornos corporativos, tales como entidades bancarias y estudios contables, donde la dispersión de la información y la dependencia de personal especializado generan cuellos de botella y sobrecarga cognitiva.

La solución técnica se despliega mediante una infraestructura desacoplada y robusta. En el backend, se implementa una arquitectura asíncrona de alto rendimiento utilizando FastAPI y Uvicorn, orquestada mediante el ecosistema LangChain. El sistema destaca por su adaptabilidad polimórfica y agnosticismos de proveedor, permitiendo la conmutación dinámica entre modelos de lenguaje comerciales en la nube (como Google Gemini 2.0 Flash) y modelos locales de código abierto (como Llama 3 o Phi-3) a través de Ollama. La persistencia y recuperación de la información combina un enfoque híbrido: el almacenamiento vectorial local embebido en disco mediante Chroma (estructurado de forma estanca por cada *tenant*) y una base de datos relacional robusta en Supabase (PostgreSQL con la extensión **pgvector**) para la gestión de usuarios, auditoría y seguridad.

Una de las principales innovaciones del núcleo computacional (**BaseModel**) radica en su enrutador inteligente de consultas. Este subsistema clasifica en tiempo real la intención del usuario para derivar la petición hacia el pipeline óptimo (búsqueda semántica RAG, consultas léxicas exactas sobre datos tabulares mediante utilidades virtuales, o navegación web), evitando el sobre costo de contexto y mejorando la precisión sintáctica. Asimismo, el procesamiento de documentos implementa estrategias avanzadas de fragmentación recursiva con solapamiento (*overlap*) para archivos PDF e ingesta estructurada mediante Pandas para documentos tabulares.

Por su parte, la interfaz de usuario se construye utilizando Next.js 15 y React, aprovechando las capacidades de los *Server Components* y el *Streaming SSR* para ofrecer una experiencia web fluida con respuestas progresivas token a token. La seguridad de la plataforma implementa una estrategia de defensa en profundidad, integrando Clerk para la autenticación multinivel (OAuth y RBAC), junto con políticas de seguridad a nivel de fila (RLS) en Supabase para garantizar el aislamiento absoluto de los datos entre clientes. Finalmente, el sistema incorpora un framework de telemetría y auditoría que registra de forma estructurada las interacciones para mitigar errores de alucinación y facilitar el análisis de precisión continuo.

Palabras clave: Generación Aumentada por Recuperación (RAG); Multi-tenancy; Inteligencia Artificial Generativa; FastAPI; Next.js 15; Base de Datos Vectorial; Gestión del Conocimiento.

1. Introducción

1.1. Contexto y motivación

En el contexto organizacional contemporáneo, los datos y el conocimiento derivados de su análisis han adquirido una relevancia estratégica equiparable a la de otros activos tradicionales. Las organizaciones los conciben como recursos clave para la mejora del desempeño, la innovación y la competitividad de las organizaciones. En ese sentido, diversos estudios coinciden en señalar que los datos dejaron de ser un subproducto operativo y se convirtieron en un activo organizacional central, cuyo aprovechamiento incide directamente en la eficiencia de los procesos, la calidad de la toma de decisiones y la capacidad de adaptación a entornos cada vez más dinámicos y complejos (Mikalef et al., 2020; Côte-Real et al., 2016).

Sin embargo, el valor de los datos no se manifiesta de manera automática ni inherente a su mera acumulación. Por el contrario, este se percibe en función de la capacidad de las organizaciones para identificar ámbitos concretos del negocio en los que los datos puedan generar impacto, planificar iniciativas analíticas coherentes con sus objetivos estratégicos y transformar la información disponible en conocimiento accionable que oriente la acción organizativa. De esta manera, el dato adquiere relevancia en la medida en que es integrado en los procesos de gestión y se articula con recursos organizacionales, capacidades analíticas y prácticas de gestión del conocimiento que permitan su uso efectivo (Mikalef et al., 2020; Côte-Real et al., 2016).

El incremento de forma sostenida de los volúmenes de información que las organizaciones deben gestionar, adicional a la creciente digitalización de procesos, servicios y relaciones organizacionales, han acabado por reforzar esta centralidad del dato. En este escenario, la disponibilidad, integridad y calidad de los datos se vinculan directamente con la continuidad del negocio y la sostenibilidad organizacional, hasta el punto de que la pérdida de acceso a la información, incluso durante períodos breves, puede representar un riesgo crítico para el funcionamiento normal de la organización (Gallego et al., 2021). De este modo, los datos y el conocimiento que se genera a partir de ellos se consolidan como activos intangibles de alto valor estratégico, difíciles de imitar y potencialmente generadores de ventajas competitivas sostenibles.

Asimismo, frente a la idea extendida de que el uso de analítica avanzada está reservado para grandes corporaciones, la evidencia muestra que este fenómeno trasciende el tamaño y el sector de la organización. Organizaciones de distintos tamaños, ámbitos de actividad y naturalezas -tanto públicas como privadas- se enfrentan al desafío de gestionar grandes volúmenes de datos y de extraer valor de ellos, lo que refuerza el carácter transversal de este fenómeno (Mikalef et al., 2020; Côte-Real et al., 2016). Esto da cuenta de que los beneficios asociados al uso de datos y analítica dependen menos del sector o del tamaño organizacional y más de la forma en que estos recursos se integran en los procesos del negocio y en las rutinas organizativas.

No obstante, pese a la creciente disponibilidad tecnológica, numerosas organizaciones aún encuentran dificultades para incorporar de manera efectiva los datos y el conocimiento en su quehacer cotidiano. Estas limitaciones no responden únicamente a problemas técnicos, sino que están estrechamente relacionadas con desafíos organizacionales y de gestión, tales como la ausencia de un lenguaje común en torno a los datos, la falta de prácticas homogéneas, las debilidades en los mecanismos de gobierno del dato y la incapacidad de gestionar adecuadamente los activos de conocimiento existentes (Mikalef et al., 2020; Gallego et al., 2021). Como consecuencia, las inversiones en infraestructuras de datos y analítica no siempre se traducen en mejoras reales del desempeño organizacional.

Así pues, los datos y el conocimiento se configuran en la actualidad como activos críticos para las organizaciones, cuyo valor se materializa únicamente cuando son gestionados de manera integral y transversal. El reto asociado a su aprovechamiento no recae exclusivamente en el ámbito tecnológico y se inscribe en una problemática organizacional más amplia, común a organizaciones de diversos sectores y tamaños, lo que justifica su análisis como un fenómeno generalizado y estructural en el entorno organizacional contemporáneo.

Reconocer a los datos y al conocimiento como activos estratégicos implica, necesariamente, asumir que su mala gestión genera impactos negativos directos sobre el desempeño organizacional. Lejos de tratarse de un problema meramente técnico, una gestión inadecuada de los datos afecta la capacidad de las organizaciones para generar

valor, sostener ventajas competitivas y tomar decisiones informadas en contextos de creciente complejidad e incertidumbre.

Diversos estudios coinciden en que el valor potencial de los datos no se materializa de forma automática, sino que depende de la existencia de capacidades organizativas, procesos y estructuras que permitan transformarlos en información útil y conocimiento accionable. Cuando estos habilitadores fallan o están ausentes, los datos pierden su condición de activo y se convierten, en muchos casos, en una fuente de ineficiencia y riesgo (Fernández Campos & Martínez Rodríguez, 2021).

Desde esta perspectiva, una gestión deficiente no solo limita la explotación analítica de los datos, sino que también incrementa los costos operativos, deteriora la confianza de los usuarios internos y externos, y compromete la sostenibilidad de la estrategia del dato en el largo plazo.

Entre las repercusiones más directas de una política de gestión de datos deficiente es la ausencia o insuficiencia de documentación: definiciones ambiguas, diccionarios incompletos, falta de trazabilidad y desconocimiento sobre el origen, significado y uso correcto de las variables. Esta carencia dificulta la interpretación de la información, incrementa la dependencia de expertos individuales y obstaculiza la reutilización del conocimiento dentro de la organización, lo que genera silos informacionales y pérdida de capital intelectual (Fernández Campos & Martínez Rodríguez, 2021).

Asimismo, la falta de un marco claro de gobernanza de datos -que establezca responsabilidades, normas de calidad, criterios de acceso y reglas de uso- deriva en inconsistencias, duplicidades y conflictos entre distintas áreas. Según el marco propuesto por Sejahtera Surbaktia et al. (2020), la ausencia de gobernanza efectiva limita la capacidad de las organizaciones para convertir grandes volúmenes de datos en valor, aun cuando dispongan de tecnologías avanzadas y capacidades analíticas formales.

La accesibilidad deficiente constituye otra consecuencia crítica. Cuando los datos no son fácilmente localizables, comprensibles o utilizables por los distintos perfiles de la organización, se restringe su aprovechamiento y se fomenta la toma de decisiones

basada en intuiciones, supuestos no validados o información parcial. Tal como señalan Fernández Campos & Martínez Rodríguez (2021), la accesibilidad es una de las palancas fundamentales del valor interno del dato: sin acceso adecuado, incluso los datos de alta calidad pierden relevancia estratégica.

En conjunto, estas deficiencias impactan negativamente en la eficiencia operativa, la calidad de los análisis, el cumplimiento normativo y la capacidad de aprendizaje organizacional, a la vez que erosionan progresivamente el valor económico y estratégico de los datos.

Si bien los problemas asociados a la gestión del conocimiento y de los datos son transversales, su manifestación concreta varía significativamente en función del tamaño, el sector y el nivel de madurez analítica de cada organización. La complejidad del ecosistema de datos, los volúmenes de información gestionados y la criticidad de los mismos introducen matices relevantes que condicionan tanto los riesgos como las posibles soluciones.

No obstante, esta heterogeneidad no invalida la existencia de patrones comunes. Por el contrario, permite observar cómo problemáticas estructurales similares emergen en contextos organizativos muy distintos, aunque con impactos y escalas diferenciadas.

En el caso de las grandes organizaciones, el principal desafío radica en la complejidad de sus ecosistemas de datos. Estas entidades gestionan volúmenes masivos de información, provenientes de múltiples sistemas, con altos requerimientos de calidad, trazabilidad y cumplimiento normativo. La coexistencia de numerosos productores y consumidores de datos, junto con arquitecturas tecnológicas heterogéneas, incrementa el riesgo de inconsistencias semánticas, redundancias y pérdida de control sobre el significado y el uso de la información (Fernández Campos & Martínez Rodríguez, 2021).

Por su parte, en organizaciones más pequeñas, los volúmenes de datos suelen ser menores y las estructuras más simples. Sin embargo, estas enfrentan limitaciones significativas en términos de recursos, formalización de procesos y especialización técnica. La gestión del conocimiento suele apoyarse en personas clave, con escasa

documentación y alta dependencia del conocimiento tácito, lo que incrementa el riesgo de pérdida de información crítica y dificulta la escalabilidad de las capacidades analíticas (Sejahtera Surbaktia et al., 2020).

A pesar de las diferencias en cantidad, naturaleza y criticidad de los datos, ambos contextos comparten problemáticas estructurales similares: falta de estandarización en las definiciones, debilidades en la gobernanza, escasa accesibilidad al conocimiento y dependencia de individuos específicos para la interpretación de la información. Estas deficiencias limitan la generación de valor a partir de los datos, independientemente del tamaño o sector de la organización.

Como lo plantean Fernández Campos y Martínez Rodríguez (2021), el valor no reside únicamente en la posesión de datos, sino en la capacidad de gestionarlos de manera sistemática a lo largo de todo su ciclo de vida. En ese sentido, tanto las grandes corporaciones como las pequeñas y medianas empresas (pymes) enfrentan el mismo desafío de fondo: transformar datos dispersos y conocimiento fragmentado en un activo organizacional accesible, confiable y reutilizable, que habilite mejores decisiones y preserve el capital intelectual de la organización en el tiempo.

1.2. Definición del problema

El valor del dato y, en particular, los inconvenientes derivados de una gestión inadecuada del conocimiento y de los activos de información, no constituyen una problemática meramente teórica. Estas limitaciones han sido observadas de manera concreta en dos organizaciones de distinto sector, tamaño y estructura organizacional, que constituyen los casos de estudio del presente Trabajo de Fin de Máster: por un lado, una de las principales instituciones financieras de España y, por otro, un estudio contable de menor escala ubicado en la ciudad de Buenos Aires, Argentina.

A pesar de las diferencias contextuales entre ambas organizaciones, un análisis preliminar permite identificar un conjunto de urgencias comunes, estrechamente alineadas con los problemas estructurales descritos en los apartados anteriores.

En primer lugar, se evidencia una alta concentración del conocimiento crítico en un número reducido de personas, generalmente perfiles senior o expertos de dominio. Esta

centralización genera cuellos de botella operativos ante las demandas cotidianas del negocio y limita la autonomía de los perfiles más jóvenes. Asimismo, la transferencia de dicho conocimiento hacia los empleados actuales y, especialmente, hacia nuevas incorporaciones, resulta insuficiente y poco sistematizada, situación que se ve agravada por el escaso tiempo destinado formalmente a procesos de capacitación. En ambos contextos, esta dependencia de individuos clave incrementa el riesgo de pérdida de conocimiento ante escenarios de rotación, desvinculación o retiro, lo que compromete la continuidad operativa y la preservación del capital intelectual de la organización.

En estrecha relación con lo anterior, se identifican barreras críticas en los procesos de incorporación y adaptación de nuevo talento (*onboarding*). La complejidad de las rutinas de trabajo, el acceso a sistemas específicos y la asimilación de la cultura organizacional exigen una transferencia de información que, al no encontrarse de manera sistematizada, prolonga la curva de aprendizaje y ralentiza la autonomía de los nuevos colaboradores. En el contexto de las pymes, por su parte, esto es visible a través de la escasa formalización de roles que obliga frecuentemente a empleados y directivos a asumir funciones polifuncionales fuera de su área de experticia. Esta falta de especialización técnica -como el caso de perfiles gerenciales que toman decisiones financieras o de recursos humanos sin el soporte documental adecuado- incrementa la probabilidad de errores operativos debido a la carencia de una fuente de consulta confiable y transversal que respalde la toma de decisiones en dominios desconocidos.

En segundo término, se identifican debilidades significativas en los mecanismos de gobernanza y documentación de los datos. Entre las principales carencias se encuentran la falta de documentación detallada sobre el funcionamiento de las tablas, la ausencia de descripciones formales de las relaciones entre entidades (*entity-relationship*), y la inexistencia de definiciones claras sobre el significado de columnas, categorías, funciones y su vinculación con otros elementos del modelo de datos. Esta problemática resulta especialmente crítica en dominios complejos y altamente regulados, como el sector asegurador o financiero, en los que la correcta interpretación semántica de los datos es un requisito indispensable para su uso efectivo. A ello se suma la ausencia de criterios homogéneos de nomenclatura, que dificulta aún más la comprensión y reutilización de los activos de información.

Otro eje problemático recurrente se vincula con la exploración y localización de la información. Frente a determinadas necesidades del negocio, tanto usuarios como desarrolladores se ven obligados a recorrer múltiples tablas, bases de datos y sistemas, e invierten un tiempo considerable no solo en identificar las fuentes potencialmente relevantes, sino también en validar su utilidad, realizar pruebas de cruces de información y gestionar solicitudes de permisos y accesos. En no pocos casos, este recorrido concluye en la constatación de que las fuentes finalmente habilitadas no resultan adecuadas para el objetivo perseguido, lo que genera una pérdida significativa de eficiencia operativa.

Asimismo, se observan ineficiencias derivadas de la ejecución manual de tareas repetitivas, como la consulta del estado de pólizas, la comparación de información proveniente de distintos sistemas o la edición manual de código para obtener datos que, en esencia, refieren al mismo objeto de negocio. Estas prácticas, además de consumir tiempo y recursos, incrementan el riesgo de errores humanos y ponen de manifiesto la ausencia de mecanismos estandarizados de acceso y consulta de la información. ponen de manifiesto la ausencia de mecanismos estandarizados de acceso y consulta de la información.

Dicha carencia de estandarización conlleva peligros críticos, como la interpretación divergente del significado de un mismo dato; esto puede derivar en cruces de información técnicamente incorrectos que, en última instancia, induzcan a la toma de decisiones erróneas basadas en premisas viciadas o incompletas. Aunado a lo anterior, la falta de una trazabilidad clara en el procesamiento manual de los datos compromete la auditabilidad de los procesos, lo que eleva el riesgo de incurrir en incumplimientos normativos o inconsistencias en los reportes regulatorios, aspecto especialmente sensible en sectores bajo estricta supervisión institucional.

Desde una perspectiva organizacional, estas limitaciones se ven reforzadas por un cambio de paradigma impulsado desde la dirección, particularmente visible en el caso de la institución financiera, en el que se demanda explícitamente la formación de “desarrolladores con mayor conocimiento de dominio”. Esta directriz busca incrementar la independencia y autonomía de los equipos técnicos para responder a las necesidades

del negocio, pero pone en evidencia la brecha existente entre dicha expectativa y las herramientas efectivamente disponibles para alcanzarla.

Por último, tanto en grandes organizaciones como en pymes, se identifican fricciones operativas asociadas a la dependencia de procesos formales y sistemas externos, como la creación de tickets en plataformas de gestión de servicios de IT (ITSM) para solicitar accesos, realizar modificaciones o resolver incidencias, lo que introduce demoras adicionales en la operativa diaria. A estas dificultades se añade la exigencia corporativa de una actualización constante frente a los cambios normativos frecuentes -como las modificaciones fiscales en la jurisdicción argentina-, lo que demanda una continua labor de búsqueda, interpretación y adaptación de la información relevante.

En conjunto, estas urgencias ponen de manifiesto que, más allá de las diferencias de escala y contexto, ambas organizaciones enfrentan desafíos estructurales similares en torno a la gestión del conocimiento, la gobernanza de datos y la accesibilidad a la información. Esta constatación refuerza la pertinencia de explorar soluciones que permitan centralizar, sistematizar y democratizar el conocimiento técnico y de negocio, a la vez que se reduce la dependencia de individuos específicos y se habilita un uso más efectivo y sostenible de los datos en la toma de decisiones.

1.3. Objetivos

A la luz de las problemáticas identificadas y de las urgencias observadas en ambos contextos organizacionales, el presente Trabajo de Fin de Máster (TFM) se plantea como un proyecto de analítica de datos orientado a diseñar, desarrollar y evaluar una solución tecnológica que permita centralizar, estructurar y democratizar el conocimiento técnico y de negocio existente en las organizaciones a través del uso intensivo de técnicas de inteligencia artificial generativa y arquitecturas avanzadas de recuperación de información.

En particular, el proyecto propone el desarrollo de una aplicación web que integre un sistema de conversación basado en inteligencia artificial, capaz de interactuar con múltiples fuentes de conocimiento estructurado y no estructurado, tales como bases de datos, documentación técnica, acuerdos normativos, diccionarios de códigos y guías

operativas. Asimismo, la plataforma está diseñada para ofrecer herramientas de control y gestión ágil, que incluyen la generación de paneles de monitoreo (*dashboards*) sobre el estado de la información, una administración sistematizada de manuales operativos y la capacidad de realizar consultas técnicas eficientes hacia esquemas de bases de datos. Todo ello bajo un esquema de optimización de recursos computacionales basado en la gestión de unidades de procesamiento de lenguaje (*tokens*), lo que garantiza la viabilidad operativa de la solución.

Así pues, la solución se concibe como un activo organizacional orientado a reducir la dependencia de individuos específicos, mejorar la accesibilidad al conocimiento y fortalecer los mecanismos de gobernanza del dato, tanto en grandes organizaciones como en pymes.

Para guiar el desarrollo de esta solución, se establecen los siguientes objetivos:

1.3.1. Objetivo general

- Diseñar y desarrollar un ecosistema integral de gestión, centralización y democratización del conocimiento organizacional que comprenda desde la recolección y estructuración de activos de información heterogéneos, hasta su posterior conversión y gestión mediante modelos de *embeddings* y grandes modelos de lenguaje (LLMs); todo ello bajo una arquitectura de inteligencia artificial generativa que permita la consulta ágil de activos técnicos y de negocio, y que garantice la estandarización, precisión y seguridad en el acceso a la información.

1.3.2. Objetivos específicos

- Sistematizar y centralizar el capital intelectual (manuales de procedimiento, normativas y conocimiento experto) en un repositorio único, de manera que las intenciones operativas del día a día sean transformadas en activos de conocimiento documentados y accesibles.
- Estandarizar los protocolos de desarrollo y reporte por medio de la definición de lineamientos comunes para códigos, hojas de cálculo y herramientas de

visualización (Power BI) que aseguren la trazabilidad, la homogeneidad estética y la correcta interpretación de la información por parte de cualquier colaborador.

- Implementar una arquitectura híbrida que combine bases de datos vectoriales para información no estructurada con una capa de conocimiento (Ontología) para representar relaciones complejas entre entidades de negocio.
- Evaluar y comparar modelos de *embeddings* y LLMs para determinar la configuración que ofrezca mayor precisión semántica en idiomas español e inglés.
- Desarrollar flujos agénticos que permitan al sistema ejecutar tareas de razonamiento multietapa, como la comparación de normativas o la interpretación de diccionarios de datos.
- Integrar un módulo de seguridad y gobernanza basado en roles, que asegure que la recuperación de información esté alineada con los permisos del usuario y la sensibilidad de los datos.
- Establecer un ciclo de retroalimentación (Feedback Loop) para el aprendizaje continuo del modelo a partir de las correcciones y valoraciones de los usuarios expertos.

1.4. Alcance y limitaciones

En últimas, el trabajo busca responder a las siguientes preguntas de investigación y de proyecto:

- ¿Cómo puede una solución basada en inteligencia artificial generativa contribuir a la preservación y transmisión del conocimiento crítico en organizaciones con distintos niveles de complejidad y madurez analítica?
- ¿De qué manera es posible estructurar y gobernar el conocimiento técnico y de negocio -incluyendo definiciones de datos, relaciones entre entidades, códigos categóricos y acuerdos operativos- para que sea accesible, reutilizable y comprensible por distintos perfiles de usuarios?
- ¿Hasta qué punto una arquitectura de recuperación aumentada por generación (RAG) permite reducir las ineficiencias asociadas a la exploración de datos, la ejecución manual de tareas repetitivas y la dependencia de expertos senior?

- ¿Cómo puede garantizarse que una solución de este tipo respete los principios de seguridad, privacidad y control de acceso a la información, especialmente en contextos altamente regulados como el sector financiero?
- ¿Qué beneficios operativos y organizacionales pueden derivarse de la implementación de una plataforma de estas características en términos de eficiencia, autonomía de los equipos y preservación del capital intelectual?

Estas preguntas orientan el diseño del proyecto y delimitan el alcance del TFM, que se sitúa en la intersección entre la analítica de datos, la gestión del conocimiento y la aplicación práctica de la inteligencia artificial. En particular, el alcance de este trabajo comprende el diseño arquitectónico, el desarrollo de una ontología de negocio y la implementación del Mínimo Producto Viable (MVP), un prototipo funcional capaz de validar la recuperación de información bajo criterios de seguridad por roles.

No obstante, el proyecto presenta ciertas limitaciones que definen su marco de ejecución:

- Volumen y representatividad de los datos: el análisis se circunscribe a un conjunto seleccionado de fuentes de las dos organizaciones en estudio. Como consecuencia de restricciones de confidencialidad y acceso, no se pretende procesar la totalidad de los activos informacionales de ambas entidades, sino aquellos que permiten validar la eficacia de la arquitectura propuesta.
- Selección y optimización de LLMs: el proyecto se ve condicionado por un compromiso entre rendimiento técnico y viabilidad económica. Debido a las restricciones de presupuesto para el uso de modelos comerciales de pago y la alta demanda de recursos de infraestructura (almacenamiento y procesamiento en la nube) que requiere el despliegue de modelos locales de gran escala, se optará por una configuración híbrida. Esta decisión implica priorizar la funcionalidad de la arquitectura sobre el rendimiento extremo de los modelos, así como aceptar las latencias y capacidades propias de los recursos disponibles.
- Evaluación de modelos de *embeddings*: en el caso de los modelos de representación vectorial, la principal limitación reside en las políticas de ciberseguridad organizacional. El uso de librerías de código abierto (*open source*), aunque permite acceder a modelos de alto rendimiento que se

aproximan a la solución ideal, suele encontrar barreras de adopción en sectores altamente regulados. Si bien existen restricciones de consumo, estas son menores en comparación con los LLMs, por lo que el alcance se centrará en validar aquellos modelos que ofrezcan el mejor equilibrio entre precisión semántica y cumplimiento de los estándares de seguridad corporativa.

- Optimización de hiperparámetros: el proyecto no contempla una exploración exhaustiva de todos los hiperparámetros disponibles en el estado del arte actual (como ajustes finos de *fine-tuning* profundo). El objetivo es identificar la configuración más eficiente dentro de un espectro representativo de modelos, sin pretender una optimización absoluta que requeriría ciclos de experimentación y cómputo fuera de los plazos académicos y presupuestarios establecidos.
- Restricciones de recursos y despliegue: la implementación se ve condicionada por el uso de recursos computacionales limitados. De haber contado con una infraestructura de mayor escala, se habría optado por el despliegue íntegro en servicios de nube de alta gama y el uso de modelos de pago con mayores capacidades de procesamiento. En consecuencia, el prototipo prioriza la viabilidad técnica sobre la velocidad de respuesta o la latencia óptima.
- Exclusiones técnicas: la orquestación mediante contenedores (Docker) y la integración en tiempo real con sistemas productivos legados quedan fuera del alcance de esta memoria, puesto que centra el esfuerzo en la lógica de recuperación, la gobernanza y la interfaz conversacional.

2. Marco teórico

2.1. Gestión de activos de información y gobernanza

2.1.1. El dato como activo estratégico

La transición hacia una economía digital ha redefinido la naturaleza de los recursos organizacionales, a la vez que ha desplazado el enfoque desde los activos físicos hacia los intangibles. En este nuevo paradigma, el dato ha dejado de ser un mero subproducto de las transacciones operativas para consolidarse como el pilar fundamental de la ventaja competitiva. Esta evolución conceptual implica una gestión de los datos orientada a su transformación en activos estratégicos con valor económico real.

Como señala Harris (2010), la capacidad de una organización para alcanzar un alto desempeño depende de la sofisticación con la que utiliza la información enterrada en sus operaciones básicas. El uso de modelos cuantitativos y analíticos permite que los datos dejen de ser registros estáticos y se conviertan en herramientas estratégicas capaces de identificar nuevas audiencias, optimizar ventas y fundamentar decisiones ejecutivas críticas. Bajo esta perspectiva, el valor no reside en la acumulación de información, sino en la capacidad analítica de la organización para extraer *insights* que permitan superar a la competencia.

No obstante, la conversión de datos en activos no es un proceso automático. Según el marco propuesto por Xu et al. (2023), existe una distinción crucial entre el "dato" y el "activo de datos" (*data asset*). Un activo de datos se define por su capacidad de ser identificado, controlado por la organización y, fundamentalmente, por su potencial para generar beneficios económicos futuros. Esta "assetización" requiere una imperativa estratégica que alinee la gestión técnica con los objetivos del negocio, lo que posibilita que la evolución conceptual del dato acompañe las demandas de la economía digital.

En la actualidad, esta visión se vuelve aún más crítica con la irrupción de tecnologías disruptivas. Caserta et al. (2023) sostienen que los datos y sus fundamentos subyacentes son los factores determinantes para el éxito de cualquier ambición en inteligencia artificial. En este sentido, el dato actúa como el "combustible central" que permite

capturar valor; sin una base de datos preparada y gobernada, las organizaciones pierden la capacidad de escalar soluciones avanzadas. Por tanto, el liderazgo de datos moderno debe evolucionar desde la simple gestión administrativa hacia un rol proactivo en el que el dato sea el motor que lidere el crecimiento y la innovación del negocio.

2.1.2. Marcos de gobernanza de datos

Una vez reconocido el dato como un activo estratégico, su gestión efectiva requiere de una estructura formal que armonice las capacidades tecnológicas con las necesidades del negocio. La gobernanza de datos se define, en este contexto, como el ejercicio de autoridad y control sobre la gestión de los activos de información, con el propósito fundamental de incrementar su valor y minimizar los costos y riesgos asociados (Abraham et al., 2019). Esta disciplina no constituye un proyecto aislado, sino un mecanismo de supervisión unificador que garantiza que los datos sean tratados como un recurso corporativo transversal (DAMA International, 2014).

Para que la gobernanza sea operativa, es imperativo establecer un marco de toma de decisiones que defina claramente quién es responsable de cada activo. Según el enfoque de Khatri y Brown (2010), el diseño de la gobernanza debe centrarse en identificar las decisiones fundamentales -tales como la definición de principios del dato, la calidad y el acceso- y asignar los derechos de decisión y responsabilidades asociados. Esta estructura de roles habilita la superación de una visión puramente administrativa para centrarse en la creación de "una única versión de la verdad", un requisito indispensable en sectores de alta complejidad como el financiero o el contable, en los que la trazabilidad y el cumplimiento normativo son críticos.

En el marco europeo, esta rigurosidad técnica se ve reforzada por exigencias legales como el Reglamento General de Protección de Datos (GDPR). Como destaca el informe de ICAP (2018), la plena aplicación de esta normativa ha transformado la gestión de la privacidad en una prioridad ineludible, pues impone restricciones estrictas y sanciones significativas para asegurar la protección de la información personal. No obstante, la implementación de estos estándares representa un desafío heterogéneo: mientras las grandes corporaciones suelen tener planes de cumplimiento avanzados, las pequeñas y

medianas empresas (pymes) exhiben mayores niveles de incertidumbre y necesidad de guía para adaptarse a estos requisitos.

Desde otra perspectiva, el auge de la inteligencia artificial ha ocasionado que la consolidación de estos marcos se vuelva aún más relevante. Como señalan Janssen et al. (2020), la gobernanza es el cimiento necesario para una IA confiable, ya que asegura que los sistemas algorítmicos se alimenten de datos consistentes, confiables y con un origen verificable. En este escenario, el concepto de linaje de datos adquiere una importancia técnica superior, al permitir rastrear el ciclo de vida de la información desde su origen hasta su consumo final por los modelos de lenguaje y, en consecuencia, mitigar sesgos e inconsistencias operativas que podrían comprometer tanto el rendimiento técnico como el cumplimiento legal.

2.1.3. Gestión del conocimiento tácito y explícito

La eficacia de una organización para innovar y mantener su competitividad no depende exclusivamente de la posesión de datos gobernados, sino de su capacidad para gestionar las distintas dimensiones del conocimiento que residen en su estructura. En este ámbito, la distinción fundamental propuesta por el modelo SECI (Socialización, Externalización, Combinación, Internalización) de Nonaka y Takeuchi identifica dos formas de conocimiento: el explícito y el tácito. Mientras que el conocimiento explícito es aquel que puede ser codificado, almacenado y transmitido formalmente en manuales o bases de datos, el conocimiento tácito representa el capital intelectual más valioso y complejo, compuesto por la experiencia, las intuiciones y el "saber hacer" acumulado por los individuos (Hoe, 2006).

La gestión de este activo intangible enfrenta desafíos críticos en el entorno organizacional contemporáneo. Como advierte O'Leary (1998), las empresas se encuentran en un escenario de vulnerabilidad constante debido a la alta rotación de personal y los procesos de reestructuración. Cuando un empleado clave abandona la entidad sin que su conocimiento haya sido capturado, la organización sufre una pérdida de memoria institucional que afecta la continuidad operativa. Por ello, el desarrollo de sistemas de gestión del conocimiento (*Knowledge Management Systems*) se vuelve imperativo para capturar este saber atomizado y hacerlo disponible de manera

transversal a todos los miembros de la organización; y, simultáneamente, para mitigar el riesgo de pérdida de información crítica ante la movilidad laboral.

La transformación de este conocimiento es un proceso dinámico y multidimensional; así como vinculado con la creación de valor organizacional. Según Nezafati et al. (2009), esta ocurre a través de la conversión constante entre el conocimiento individual y el colectivo, y entre lo tácito y lo explícito. Este flujo requiere no solo de la acumulación de "volumen de conocimiento", sino de una velocidad de transformación adecuada que permita que las experiencias individuales se conviertan en documentación técnica estandarizada y reutilizable. En ese sentido, la estandarización actúa como el puente necesario para que el conocimiento deje de depender de expertos aislados y se convierta en un recurso organizacional democratizado.

En última instancia, el éxito de una arquitectura tecnológica orientada a la democratización del saber -como la propuesta en este trabajo- reside en su capacidad para facilitar estos procesos de socialización e internalización. Al formalizar el conocimiento experto en repositorios estructurados, la organización no solo preserva su capital intelectual frente a la fuga de talento, sino que habilita una base de conocimiento robusta que sirve como cimiento para la implementación de soluciones de inteligencia artificial capaces de razonar sobre la experiencia acumulada de la organización.

2.2. Inteligencia artificial generativa y LLMs

2.2.1. Evolución de los modelos de lenguaje

El procesamiento del lenguaje natural (PLN) ha experimentado una evolución histórica dividida en fases que reflejan el avance de la capacidad computacional y la complejidad algorítmica. En sus etapas iniciales, el campo estuvo dominado por modelos simbólicos y estadísticos basados en reglas gramaticales complejas y en el cálculo de probabilidades de secuencias de palabras o n-gramas (Khurana et al., 2023). Estos enfoques, aunque fundamentales, presentaban limitaciones significativas para captar la semántica profunda y las dependencias de largo alcance en textos extensos, debido a que el lenguaje era tratado como una serie de eventos discretos y locales.

Con la irrupción del aprendizaje profundo (*deep learning*), el paradigma se desplazó hacia el uso de redes neuronales recurrentes (RNN) y redes de memoria corta y larga (LSTM). Como explican Abraham et al. (2020), estos modelos permitieron una representación más fluida del lenguaje al procesar la información de manera secuencial; sin embargo, enfrentaban dificultades críticas como el desvanecimiento del gradiente y la imposibilidad de paralelizar el entrenamiento, lo que limitaba su escalabilidad ante volúmenes masivos de datos. La historia del PLN demuestra que cada avance ha buscado superar la barrera de la "memoria" y la comprensión del contexto global del mensaje.

El punto de inflexión definitivo en esta evolución ocurrió con la propuesta de la arquitectura *Transformer*. A diferencia de sus predecesores recurrentes, este modelo prescinde de la secuencialidad y se basa exclusivamente en mecanismos de atención. Según el trabajo seminal de Vaswani et al. (2017), el mecanismo de "auto-atención" (*self-attention*) permite que el modelo evalúe de manera simultánea la relevancia de cada palabra en relación con todas las demás dentro de una oración, sin importar la distancia física entre ellas. Esta capacidad no solo mejoró drásticamente la calidad de la traducción y la generación de texto, sino que permitió un entrenamiento significativamente más rápido y eficiente al aprovechar el procesamiento en paralelo.

Esta transición hacia los *Transformers* ha sentado las bases de los modelos de lenguaje modernos, y ha posibilitado que la IA deje de simplemente predecir la siguiente palabra en función del uso de estadísticas de frecuencia, para pasar a una comprensión contextual rica y multidimensional. La eliminación de la dependencia de la recurrencia ha permitido el desarrollo de modelos con miles de millones de parámetros, capaces de capturar matices lingüísticos complejos que antes resultaban inalcanzables para los sistemas estadísticos tradicionales.

2.2.2. Large Language Models (LLMs)

Los Modelos de Lenguaje de Gran Escala (LLMs) constituyen una evolución masiva de la arquitectura *Transformer*, caracterizada por un incremento exponencial en el número de parámetros y el volumen de datos de entrenamiento. Estos modelos han demostrado capacidades emergentes que trascienden el simple procesamiento de texto, en la medida

en que son capaces de solucionar tareas complejas de razonamiento y generación de código que no fueron explícitamente programadas (Minaee et al., 2024). Su arquitectura les permite actuar como modelos fundacionales, que hacen las veces de base para una amplia variedad de aplicaciones mediante el aprendizaje de representaciones lingüísticas universales.

El desarrollo de un LLM se fundamenta en un proceso de entrenamiento en etapas. La fase inicial, denominada pre-entrenamiento, consiste en exponer al modelo a corpus masivos de datos -a menudo de escala superior al terabyte- para que aprenda a predecir el siguiente token en una secuencia. Este aprendizaje auto-supervisado permite al modelo internalizar la estructura del lenguaje y un vasto conocimiento general sobre el mundo (Zhao et al., 2023). Posteriormente, se aplican técnicas de ajuste fino (*fine-tuning*) para especializar al modelo en dominios específicos o mejorar su capacidad de seguir instrucciones. Un aspecto crítico en esta fase es la robustez del modelo; según Chen et al. (2020), el entrenamiento adversario durante el ajuste fino no solo mejora la precisión en tareas específicas, sino que fortalece al modelo frente a entradas diseñadas para confundirlo y, por tanto, optimizar su estabilidad general.

Un factor determinante en la operatividad de estos sistemas es la ventana de contexto (*context window*). Este parámetro define la cantidad máxima de información que el modelo puede "observar" y mantener en memoria de trabajo durante una sola inferencia (Douglas, 2023). Debido a la complejidad computacional de los mecanismos de atención, esta ventana actúa como una limitación física: si el input -como un manual técnico extenso o una base de datos de auditoría- excede este límite, el modelo pierde la capacidad de relacionar fragmentos distantes del texto. La optimización de esta ventana es esencial para aplicaciones que requieren procesar documentos largos sin perder la coherencia semántica.

Finalmente, el despliegue de LLMs en entornos corporativos enfrenta el desafío de las alucinaciones. Este fenómeno se refiere a la generación de respuestas que, aunque gramaticalmente impecables y seguras en su tono, carecen de veracidad fáctica o lógica (Zhao et al., 2023). Las alucinaciones son una consecuencia intrínseca de la naturaleza probabilística de estos modelos, los cuales no "consultan" una base de datos de hechos, sino que calculan la secuencia de palabras más probable. Para mitigar este riesgo, es

necesario implementar estrategias que anclen la generación de texto a fuentes externas verificables para asegurar que la salida del modelo sea no solo coherente, sino también precisa y confiable dentro del marco regulatorio de la organización.

2.2.3. Modelos de *embeddings*

La capacidad de los sistemas de inteligencia artificial para procesar el lenguaje natural de manera efectiva reside en la transformación de unidades lingüísticas en representaciones numéricas densas. Los modelos de *embeddings* son estrategias que construyen vectores de longitud fija en un espacio multidimensional, y se basan en la hipótesis distribucional, que sugiere que las palabras que aparecen en contextos similares tienden a tener significados similares (Almeida & Xexéo, 2019). A diferencia de las representaciones antiguas y dispersas (como *one-hot encoding*), los *embeddings* capturan relaciones semánticas y sintácticas complejas, a la vez que permiten que conceptos relacionados se ubiquen en puntos cercanos dentro del espacio vectorial.

En la práctica, estas representaciones vectoriales permiten ir más allá de la simple coincidencia de palabras clave. Según Bakarov (2018), los *embeddings* funcionan como representaciones de valor real que capturan la semántica léxica entrenada en grandes corpus. Esta característica es fundamental para la recuperación de información, ya que permite al sistema identificar la relevancia de un documento incluso si este no contiene las palabras exactas de la consulta, sino términos semánticamente equivalentes. La evaluación de estos modelos se divide en métodos intrínsecos, que miden la coherencia de las relaciones entre vectores, y extrínsecos, que validan su rendimiento en tareas finales como la clasificación o la búsqueda (Bakarov, 2018).

Para determinar la relevancia entre una consulta y un documento en este espacio vectorial, se utiliza predominantemente la similitud coseno. Este algoritmo mide el coseno del ángulo entre dos vectores; cuanto más pequeño es el ángulo, mayor es el valor del coseno y, por ende, mayor es la similitud semántica entre los textos (Darmalaksana et al., 2020). Este enfoque es particularmente robusto en motores de búsqueda semántica, ya que no depende de la longitud de los documentos, sino de la dirección u orientación de sus vectores representativos. Como señalan Rifai & Anugrah (2021), la implementación de la similitud coseno permite superar las limitaciones de los

motores de búsqueda tradicionales que solo comparan títulos o palabras clave literales, facilitando el hallazgo de información precisa en estructuras complejas como artículos científicos o bases de conocimiento técnico.

En el contexto de esta investigación, los modelos de *embeddings* constituyen el puente técnico que permite al sistema RAG convertir los documentos de gobernanza y manuales técnicos en una base de datos vectorial consultable. La búsqueda por proximidad semántica asegura que el modelo de lenguaje reciba el contexto más pertinente para responder a las consultas de los usuarios, minimizando el ruido informativo y optimizando la precisión de las respuestas generadas.

2.3. Arquitecturas de Recuperación de Información (RAG)

2.3.1. Fundamentos de RAG (Retrieval-Augmented Generation)

La técnica de Generación Aumentada por Recuperación (RAG) surge como una arquitectura híbrida diseñada para optimizar la precisión y la fiabilidad de los modelos de lenguaje de gran escala (LLM). El concepto fundamental de RAG es permitir que el modelo funcione bajo un esquema de "libro abierto": en lugar de depender exclusivamente del conocimiento estático comprimido en sus parámetros durante el entrenamiento (conocimiento paramétrico), el sistema consulta dinámicamente una base de conocimientos externa y actualizada (conocimiento no paramétrico) antes de generar una respuesta (Lewis et al., 2021).

Desde una perspectiva técnica, el proceso RAG se descompone en dos fases principales que ocurren en tiempo de ejecución. Primero, el componente de recuperación (*retrieval*) utiliza una consulta del usuario para identificar y extraer los fragmentos de documentos más relevantes de un corpus externo, típicamente almacenados en una base de datos vectorial. Segundo, el componente de generación toma tanto la consulta original como los fragmentos recuperados para producir una respuesta coherente y fundamentada (Gao et al., 2023). Este enfoque permite que el modelo tenga acceso a información que no estaba disponible durante su fase de pre-entrenamiento, como documentos internos de una organización o noticias de última hora.

De acuerdo con IBM (2024), esta arquitectura aborda tres de los desafíos más críticos en el despliegue de IA generativa:

1. Reducción de alucinaciones: al obligar al modelo a basar su respuesta en hechos recuperados, se minimiza la generación de información falsa.
2. Actualización de conocimientos: permite actualizar la base de conocimientos sin necesidad de costosos procesos de re-entrenamiento del modelo base.
3. Transparencia y citación: el sistema puede proporcionar referencias directas a las fuentes de donde extrajo la información, permitiendo la verificación por parte del usuario.

Sin embargo, el paradigma RAG está evolucionando hacia modelos más dinámicos. Mientras que los sistemas estándar realizan una única recuperación al inicio, propuestas más avanzadas como la de Jiang et al. (2023) sugieren una "recuperación activa" (*active retrieval*). En este escenario, el modelo decide cuándo y qué buscar a lo largo de la generación de textos largos, anticipando las necesidades de información futuras para evitar la degradación de la calidad en respuestas extensas. Esta capacidad de consulta iterativa es esencial para tareas complejas que requieren múltiples pasos de razonamiento o la integración de diversas fuentes de datos.

En el marco de este TFM, el uso de RAG actúa como el motor de confianza del sistema, asegurando que las respuestas proporcionadas al personal técnico o de auditoría no sean meras inferencias estadísticas, sino deducciones basadas estrictamente en la normativa y manuales vigentes de la institución.

2.3.2. Estrategias de *chunking* y recuperación

En los sistemas de Generación Aumentada por Recuperación (RAG), el proceso de segmentación de documentos o *chunking* constituye un paso crítico que determina la calidad de la información recuperada. Dado que los modelos de lenguaje tienen una ventana de contexto limitada y los modelos de *embeddings* pierden precisión al procesar secuencias excesivamente largas, es imperativo dividir los documentos originales en fragmentos o "chunks" manejables. Este proceso busca un equilibrio delicado: fragmentos demasiado pequeños pueden carecer del contexto necesario para ser útiles,

mientras que fragmentos demasiado extensos pueden introducir ruido semántico e incrementar los costos computacionales (Qu et al., 2024).

Las estrategias de *chunking* han evolucionado desde métodos ingenuos hacia enfoques conscientes del contenido. Según Zhou et al. (2026), las estrategias se pueden clasificar en dos grandes paradigmas:

1. Segmentación basada en estructura: utiliza elementos físicos del documento como longitud de caracteres fija, oraciones o párrafos. Aunque son eficientes y fáciles de implementar, a menudo rompen la continuidad de una idea a mitad de una frase.
2. Segmentación semántica: utiliza modelos de inteligencia artificial para identificar puntos de ruptura donde el significado del texto cambia. Esto asegura que cada fragmento sea un bloque de información lógicamente completo.

Investigaciones recientes subrayan que el *chunking* consciente de la estructura, específicamente el "Paragraph Group Chunking", ha demostrado ser una de las técnicas más efectivas, logrando tasas de acierto (*Hit Rate*) y precisión significativamente superiores a la segmentación simple por caracteres (Shaukat et al., 2026). Este método agrupa párrafos relacionados, preservando la jerarquía y la intención del autor, lo cual es vital al procesar manuales técnicos o documentos de gobernanza donde la estructura (títulos, subtítulos y listas) contiene carga semántica implícita.

No obstante, la implementación de técnicas avanzadas como el *chunking* semántico conlleva un debate sobre la eficiencia. Aunque estas estrategias mejoran la coherencia de los segmentos, algunos estudios sugieren que el alto costo computacional de realizar inferencias adicionales para segmentar el texto no siempre se traduce en una ganancia de rendimiento proporcional frente a un *chunking* de tamaño fijo bien optimizado con solapamiento (*overlap*) (Qu et al., 2024). Por otro lado, en contextos educativos o de diseño instruccional, la segmentación automática no solo sirve para la recuperación, sino que permite generar metadatos descriptivos y objetivos de aprendizaje específicos para cada fragmento, facilitando la navegabilidad de grandes repositorios de datos (Tran et al., 2018).

Para este proyecto, la elección de la estrategia de recuperación no es trivial. La sensibilidad del modelo de *embeddings* al tamaño del fragmento sugiere que se debe emplear una técnica adaptativa o basada en párrafos, garantizando que el sistema RAG identifique con precisión los controles y normativas bancarias sin perder el contexto legal que los rodea.

2.3.3. Bases de datos vectoriales vs. Bases de datos relacionales

La integración de capacidades de búsqueda semántica en las arquitecturas organizacionales ha generado un debate técnico sobre la infraestructura subyacente más adecuada: el uso de bases de datos vectoriales especializadas (Pure Vector Databases) o la adaptación de bases de datos relacionales generalistas (Generalized Vector Databases). Esta distinción es fundamental, ya que los sistemas relacionales tradicionales (como PostgreSQL) fueron diseñados para datos estructurados y consultas exactas, mientras que los sistemas vectoriales están optimizados para el manejo de representaciones numéricas de alta dimensionalidad y búsquedas de similitud (Taipalus, 2024).

Las bases de datos vectoriales especializadas, como Milvus o Pinecone, operan bajo un paradigma donde el vector es el ciudadano de primera clase. Según Taipalus (2024), estos sistemas están diseñados desde cero para abordar los desafíos únicos de los datos vectorizados, tales como la alta dimensionalidad y la dispersión (sparsity). Su principal ventaja reside en la implementación de índices de Búsqueda de Próximos Vecinos Aproximados (ANN), que permiten realizar consultas de similitud de manera extremadamente eficiente en milisegundos, incluso sobre volúmenes de datos a escala de miles de millones. Sin embargo, estas bases de datos suelen carecer de la madurez y las garantías de consistencia (propiedades ACID) que ofrecen los sistemas relacionales, y a menudo requieren que los metadatos se gestionen de forma externa, lo que incrementa la complejidad de la arquitectura (Taipalus, 2024).

Por otro lado, las bases de datos relacionales generalizadas han intentado cerrar esta brecha mediante extensiones (como pgvector para PostgreSQL) que permiten almacenar vectores en columnas y realizar búsquedas de similitud mediante operadores específicos. No obstante, investigaciones recientes sugieren que existen limitaciones

fundamentales en este enfoque. Zhang et al. (2024) identifican que, aunque PostgreSQL es altamente versátil, enfrenta cuellos de botella críticos al procesar datos vectoriales debido a su arquitectura basada en páginas y su gestión de memoria. Sus experimentos demuestran que, en escenarios de alta carga, las bases de datos relacionales pueden exhibir un rendimiento significativamente inferior al de los sistemas especializados, especialmente en lo que respecta a la latencia de consulta y la velocidad de indexación (Zhang et al., 2024).

A pesar de las ventajas de rendimiento de los sistemas especializados, Zhang et al. (2024) subrayan que el uso de bases de datos relacionales puede ser preferible para organizaciones que ya cuentan con una infraestructura de datos consolidada y requieren una integración profunda entre los vectores y los metadatos relacionales. La decisión entre una u otra arquitectura depende, por tanto, del equilibrio entre la necesidad de rendimiento bruto en la búsqueda semántica y la necesidad de mantener la integridad y simplicidad operativa que ofrecen los sistemas relacionales tradicionales.

Para el propósito de este trabajo, se analizará cómo la elección de la base de datos influye en la capacidad del sistema RAG para recuperar normativas con precisión, considerando que en el sector financiero la consistencia de los datos y la trazabilidad de los metadatos (como autores o fechas de vigencia) son tan críticos como la similitud semántica en sí misma.

2.4. Ingeniería del Conocimiento e IA Híbrida

2.4.1. Ontologías y Grafos de Conocimiento

La representación de relaciones lógicas complejas en los sistemas de información modernos ha evolucionado desde los esquemas relacionales rígidos hacia estructuras más flexibles y expresivas basadas en la web semántica. En este contexto, las ontologías se definen como especificaciones formales y explícitas de una conceptualización compartida, proporcionando el vocabulario y las reglas lógicas necesarias para describir un dominio de conocimiento (Yahya et al., 2021). Su función principal es permitir que las máquinas no solo almacenen datos, sino que comprendan el significado de las relaciones entre ellos, facilitando la interoperabilidad y el razonamiento automatizado.

Los Grafos de Conocimiento (Knowledge Graphs) actúan como la implementación práctica de estas ontologías a gran escala. Según Hogan et al. (2021), un grafo de conocimiento adquiere y define información sobre entidades del mundo real mediante una estructura de red, donde los nodos representan objetos o conceptos y las aristas representan las relaciones entre ellos. A diferencia de las bases de datos tradicionales, los grafos permiten representar relaciones transversales complejas (como la unión lógica entre tablas de diferentes dominios) de forma nativa. Esto es posible gracias al uso de modelos de datos basados en grafos dirigidos y etiquetados, comúnmente expresados en el marco de descripción de recursos (RDF).

La integración de estas tecnologías permite resolver problemas de semántica que los sistemas relacionales no pueden abordar por sí solos. D'Amato et al. (2017) destacan que la Web Semántica proporciona los estándares necesarios (como OWL y SPARQL) para que los datos sean legibles por máquinas y para que las relaciones lógicas —por ejemplo, determinar que la "Tabla A se une con la Tabla B" debido a una dependencia de negocio específica— se conviertan en hechos explícitos y consultables. En entornos industriales y corporativos, el uso de grafos de conocimiento facilita la gestión de activos al permitir una visión holística que conecta datos técnicos, procesos de negocio y requisitos regulatorios en una única estructura de red (Yahya et al., 2021).

En resumen, mientras que las bases de datos vectoriales se enfocan en la similitud estadística, los grafos de conocimiento aportan la lógica de negocio y el rigor semántico. Como señala Hogan et al. (2021), la potencia de estos grafos reside en su capacidad para soportar razonamientos deductivos, lo que permite inferir nueva información a partir de las relaciones ya existentes, proporcionando un contexto estructurado que es vital para la precisión de los sistemas de inteligencia artificial.

2.4.2. GraphRAG

La técnica de Generación Aumentada por Grafos (GraphRAG) representa la intersección avanzada entre la recuperación de información y el razonamiento estructurado. Mientras que el RAG convencional se limita a la búsqueda por similitud semántica (basada en vectores), GraphRAG estructura el conocimiento externo como grafos de conocimiento multiescala, lo que permite a los modelos de lenguaje (LLM)

integrar tanto el contexto global de un corpus como los detalles granulares de las entidades (Liang et al., 2025). Esta arquitectura es especialmente valiosa para resolver consultas que requieren una lógica de negocio compleja, superando la simple coincidencia de palabras clave.

Un avance significativo en esta disciplina es el enfoque de "Local a Global" propuesto por Edge et al. (2024). Según estos autores, el RAG tradicional a menudo falla al responder preguntas globales sobre un corpus completo (por ejemplo, "¿cuáles son los temas principales de esta normativa?"), ya que está diseñado para la recuperación de fragmentos específicos. GraphRAG soluciona esto mediante un proceso de dos etapas: primero, utiliza un LLM para construir un índice de grafos a partir de documentos privados; segundo, genera resúmenes de comunidades de entidades. Esto permite que el sistema responda preguntas que requieren una síntesis de toda la colección de documentos, una tarea que antes resultaba inalcanzable para los sistemas basados únicamente en vectores (Edge et al., 2024).

Desde una perspectiva de arquitectura, GraphRAG aprovecha la naturaleza intrínseca de los grafos (nodos conectados por aristas) para codificar información heterogénea y relacional masiva. Como señalan Han et al. (2025), la capacidad de equipar a los modelos de lenguaje con grafos los convierte en un "recurso dorado" para aplicaciones del mundo real que dependen de relaciones lógicas explícitas. No obstante, esta potencia también introduce nuevos desafíos de seguridad. Liang et al. (2025) advierten sobre la vulnerabilidad a ataques de "envenenamiento" de grafos, donde la inyección de relaciones falsas puede comprometer múltiples consultas simultáneamente al explotar las relaciones compartidas dentro del grafo.

En conclusión, la implementación de GraphRAG en este proyecto permite que el sistema no solo encuentre "párrafos similares", sino que sea capaz de navegar por la estructura lógica de la gobernanza de datos. Al utilizar bases de datos de grafos (como Neo4j), el sistema puede realizar razonamientos de múltiples saltos (*multi-hop reasoning*), conectando controles normativos con activos de información específicos, garantizando así que la respuesta generada esté alineada con la lógica operativa y no solo con la frecuencia estadística de los términos.

2.5. Paradigma de Agentes de IA y Orquestación

2.5.1. Agentes Inteligentes

La evolución de los modelos de lenguaje de gran escala (LLM) ha trascendido la mera generación de texto para dar paso a los agentes inteligentes autónomos. Estos sistemas se definen como entidades capaces de percibir su entorno, razonar sobre objetivos complejos y ejecutar acciones para interactuar con el mundo exterior (Wang et al., 2024). A diferencia de un LLM estándar, que opera como un sistema pasivo de respuesta, un agente utiliza el modelo como un "cerebro" central que coordina procesos de planificación y memoria para resolver tareas de manera autónoma.

El núcleo operativo de estos agentes reside en su capacidad de razonamiento y planificación. Según Wang et al. (2024), los agentes descomponen tareas complejas en subtareas manejables, utilizando estrategias como el pensamiento de cadena (*Chain-of-Thought*) para reflexionar sobre los pasos necesarios antes de actuar. Esta capacidad de "auto-corrección" permite que el agente evalúe si la información recuperada es suficiente o si debe buscar recursos adicionales, mitigando limitaciones intrínsecas del modelo como las alucinaciones o la falta de datos actualizados (Jin et al., 2025).

Un componente distintivo de los agentes modernos es el aprendizaje y uso de herramientas (*tool learning*). Como señalan Xu et al. (2025), los agentes pueden ser equipados con un conjunto de herramientas externas (APIs, motores de búsqueda o bases de datos) para ampliar sus capacidades más allá de sus datos de entrenamiento. El agente no solo utiliza la herramienta, sino que debe "aprender" a seleccionarla: ante una duda técnica, el agente decide si debe consultar un manual interno mediante RAG o si requiere realizar una consulta SQL a una base de datos operativa. Este proceso implica un ciclo de retroalimentación donde el agente observa el resultado de la herramienta y ajusta su plan de acción en consecuencia (Xu et al., 2025).

Finalmente, la efectividad de estos agentes se mide a través de marcos de evaluación que analizan dimensiones como la robustez, la eficiencia de costes y la seguridad (Yehudai et al., 2026). En entornos corporativos, donde el agente tiene acceso a activos

de información sensibles, es crítico asegurar que el proceso de toma de decisiones sea trazable. La capacidad de los agentes para razonar sobre "qué herramienta usar" no solo optimiza la precisión de la respuesta, sino que permite una orquestación inteligente de los recursos de conocimiento de la organización, convirtiendo al sistema en un colaborador proactivo en tareas de auditoría y gestión de activos (Jin et al., 2025).

2.5.2. Frameworks de orquestación

La construcción de sistemas basados en agentes requiere de una infraestructura que coordine la interacción entre los modelos de lenguaje, las bases de datos y las herramientas externas. Los frameworks de orquestación han surgido para estandarizar estos flujos de trabajo, permitiendo a los desarrolladores pasar de simples llamadas a una API a la creación de tuberías (*pipelines*) complejas y multi-etapa (Sapkota et al., 2025). Estos marcos de trabajo actúan como el pegamento lógico que gestiona el estado, la memoria y la lógica de control del sistema.

Dentro del ecosistema actual, se distinguen dos paradigmas principales de diseño:

1. Composición basada en cadenas: representada por librerías como LangChain, se enfoca en secuencias lineales de acciones donde la salida de un componente es la entrada del siguiente. Este enfoque es altamente eficiente para tareas de RAG estándar, pero puede presentar limitaciones en flujos que requieren ciclos o una lógica de control altamente adaptativa (Sapkota et al., 2025).
2. Orquestación basada en grafos de estado: herramientas como LangGraph introducen la capacidad de crear flujos cíclicos y persistentes. Al tratar el proceso como un grafo donde cada nodo es una acción y las aristas definen la transición basada en el estado actual, se facilita la creación de agentes que pueden iterar, reflexionar sobre sus propios errores y mantener una memoria a largo plazo más robusta (Kaplunovich, 2025; Sapkota et al., 2025).

Un aspecto crítico en la orquestación moderna es la observabilidad y el control de flujo. A medida que los agentes ganan autonomía, aumenta el riesgo de ejecuciones infinitas o costes imprevistos. Frameworks como LlamaIndex y extensiones como LangSmith proporcionan capas de telemetría esenciales para monitorizar la latencia, el uso de

tokens y el rastreo (*tracing*) de decisiones, permitiendo auditar por qué un agente decidió utilizar una herramienta específica en un momento dado (Kaplunovich, 2025). Esta transparencia es indispensable en el sector financiero para garantizar que el sistema de gestión de activos cumpla con los estándares de gobernanza.

Finalmente, la tendencia actual se dirige hacia arquitecturas de multi-orquestación, donde diferentes motores coordinan múltiples LLM especializados. Según Rasal (2024), estos sistemas híbridos pueden integrar bases de datos vectoriales con bases de datos de grafos temporales para capturar no solo información estática, sino también la evolución de las preferencias del usuario y el contexto de la conversación. Esta orquestación avanzada permite que el sistema actúe como un asistente contextualmente rico y personalizado, capaz de sintetizar respuestas complejas a partir de fuentes de datos heterogéneas (Rasal, 2024; Roman & Roman, 2026).

2.6. Seguridad, privacidad y ética en la IA corporativa

2.6.1. Control de Acceso Basado en Roles (RBAC) en IA

La integración de modelos de lenguaje de gran escala (LLM) en entornos corporativos introduce desafíos críticos en la gestión de la seguridad, ya que los métodos tradicionales de control de acceso suelen ser insuficientes para manejar la naturaleza generativa de la IA. El Control de Acceso Basado en Roles (RBAC) en sistemas de IA se define como la capacidad de filtrar y condicionar el comportamiento y las respuestas del modelo según los privilegios asociados a distintos roles organizacionales (Almheiri et al., 2024). Esto es fundamental para garantizar que, por ejemplo, un usuario con nivel de acceso "Silver" no reciba información confidencial que solo debería estar disponible para un nivel "Gold".

A pesar de su amplia adopción por su simplicidad, el RBAC estático presenta dificultades para adaptarse a comportamientos de usuario dinámicos y riesgos de seguridad en evolución. Según Feng et al. (2026), la integración de LLM y RAG permite evolucionar desde roles estáticos hacia decisiones conscientes del contexto. Estos sistemas avanzados no solo verifican la identidad del usuario, sino que analizan el contenido de la consulta y el contexto operativo para aplicar políticas de acceso de

manera más flexible y precisa. Esto permite que el sistema de IA actúe como un guardián inteligente que interpreta las políticas de seguridad en tiempo real antes de generar una respuesta (Feng et al., 2026).

La implementación técnica de estas restricciones puede abordarse mediante diversas estrategias, como el ajuste fino del modelo condicionado al rol (*role-conditioned generation*) o el uso de clasificadores intermedios que validan las solicitudes antes de que el LLM las procese (Almheiri et al., 2024). Además, los LLM están demostrando ser herramientas eficaces para la síntesis automática de políticas de acceso, facilitando la creación de reglas complejas que, de otro modo, serían propensas a errores humanos en sistemas de computación en la nube (Vatsa et al., 2025).

Finalmente, la seguridad del control de acceso en IA debe ser robusta frente a amenazas específicas como la inyección de prompts (*prompt injection*) y intentos de evasión (*jailbreak*). Como destacan Xu et al. (2024) en su revisión sistemática, la ciberseguridad asistida por LLM requiere una vigilancia constante de las vulnerabilidades en los contratos inteligentes y las interfaces de gestión de datos. En este sentido, un sistema de RBAC bien diseñado en una arquitectura RAG no solo limita el acceso a documentos, sino que garantiza que el razonamiento del modelo permanezca dentro de los límites de la gobernanza de datos de la organización (Almheiri et al., 2024; Xu et al., 2024).

2.6.2. Soberanía del dato

La soberanía del dato se ha convertido en un pilar crítico para las organizaciones que adoptan modelos de lenguaje de gran escala (LLM), especialmente en sectores con regulaciones estrictas de privacidad. Este concepto refiere al derecho y la capacidad de una organización para mantener el control total sobre dónde se almacenan y procesan sus datos, evitando que información sensible sea expuesta a proveedores de servicios en la nube externos (Radas et al., 2025). La elección entre el uso de APIs comerciales (como las de OpenAI o Google) y el despliegue de modelos en infraestructura propia (*on-premise*) define el equilibrio entre facilidad de uso, coste y seguridad.

El despliegue *on-premise* surge como la solución principal para garantizar la privacidad absoluta. Según Radas et al. (2025), soluciones como UniGPT permiten a las

instituciones ejecutar modelos de código abierto (como Llama o Mistral) en servidores internos, asegurando que los datos de entrenamiento y las consultas de los usuarios nunca abandonen la red institucional. Este enfoque mitiga riesgos críticos descritos por Li et al. (2024), como la fuga de datos a través de las consultas de los usuarios o el uso no autorizado de información corporativa para el reentrenamiento de modelos comerciales. No obstante, el despliegue local exige una inversión significativa en hardware especializado (GPUs) y una ingeniería de sistemas robusta para gestionar la latencia y la concurrencia (Pan et al., 2025).

Como alternativa eficiente, las arquitecturas de nube híbrida proponen un modelo de colaboración entre el borde (*edge*) y la nube. Xin (2024) describe un sistema donde un modelo ligero local procesa la mayoría de las solicitudes rutinarias, manteniendo los datos sensibles dentro de la organización, mientras que solo las consultas complejas y no sensibles se derivan a modelos potentes en la nube mediante mecanismos de enrutamiento basados en la confianza. Este enfoque no solo mejora la soberanía del dato al filtrar qué información sale de la red, sino que reduce los costes operativos de las APIs en más de un 60% y disminuye la latencia promedio (Xin, 2024).

Desde el punto de vista de la seguridad, la soberanía del dato es una defensa proactiva contra ataques de extracción de datos. Yan et al. (2024) subrayan que el procesamiento local es una de las defensas más efectivas contra ataques de inferencia y extracción de información sensible, ya que elimina el vector de ataque que representa la transmisión de datos a través de internet. En conclusión, la viabilidad económica del despliegue local frente a la nube depende del volumen de consultas y la sensibilidad de los activos de información; para organizaciones que gestionan datos críticos, el control soberano sobre el ciclo de vida del dato justifica la complejidad técnica del despliegue en infraestructura propia (Pan et al., 2025; Yan et al., 2024).

3. Propuesta

Esta sección pretende detallar el trabajo realizado, el cual ha sido concebido a través de dos etapas fundamentales: el diseño del sistema y su implementación técnica. El primer componente abarca la arquitectura de la solución, los casos de uso corporativos que otorgan contexto al sistema, la gestión multimodal y multilingüe de diversos tipos de archivos y lenguajes, y la lógica del flujo agéntico; es decir, la implementación del razonamiento para la toma de decisiones. Desde la perspectiva de la implementación técnica, se detalla el *stack* tecnológico, el desarrollo del núcleo del modelo - basado en la clase *BaseModel* y sus adaptaciones específicas -, la creación del entorno de usuario por medio de una interfaz y plataforma *Software as a Service* (SaaS) y, en último lugar, la gestión de configuraciones, delineada por la política de acceso y control de datos.

3.1. Metodología y diseño del sistema

La metodología seguida para el diseño de esta solución descansa en la fundamentación de un enfoque iterativo centrado en la recuperación de información de distinto tipo y el razonamiento lógico. En este apartado se describe la estructura conceptual que permite al sistema procesar consultas complejas de manera eficiente. El diseño no contempla únicamente la disposición técnica de los componentes (arquitectura), sino que se articula en torno a la necesidad de resolver problemáticas específicas en entornos empresariales mediante el uso de arquitecturas RAG. A continuación, se detallan los pilares que sostienen esta propuesta, desde el flujo de datos inicial hasta los mecanismos de decisión agéntica que garantizan la precisión de las respuestas.

3.1.1. Arquitectura del sistema: diagrama de bloques del flujo de datos

La arquitectura de la solución ha sido concebida bajo un esquema modular, entendido como un diseño basado en la división del sistema en partes más pequeñas, independientes y funcionalmente bien definidas. Esta modularidad garantiza que cada componente cumpla una tarea específica, lo que permite que eventuales modificaciones o actualizaciones en un módulo no afecten la estabilidad del resto del sistema, siempre que se preserven las interfaces de comunicación. Asimismo, la solución opera bajo un esquema agéntico, que faculta al sistema para procesar la información de manera

dinámica y autónoma, así como para adaptarse a la naturaleza y complejidad de la consulta del usuario.

A diferencia de las estructuras lineales rígidas, esta arquitectura funciona como un ecosistema de componentes interconectados que priorizan la precisión mediante un flujo de datos estructurado en cuatro etapas principales: (i) recepción y clasificación, (ii) procesamiento de contexto y recuperación, (iii) síntesis y razonamiento, y (iv) entrega de respuesta y trazabilidad.

(i) Recepción y clasificación (capa de conocimiento)

El flujo se inicia cuando el usuario realiza una consulta en lenguaje natural. Esta es recibida por el núcleo del sistema, representado por la clase BaseModel, que actúa como orquestador central. En esta etapa, el LLM ejecuta una función de enrutamiento lógico (*Router*). Mediante la aplicación de un *prompt* de clasificación estricto, el sistema analiza la intención de la pregunta y decide la ruta óptima de ejecución entre las siguientes opciones: búsqueda semántica en documentos (RAG), consulta en bases de datos estructuradas (tablas), respuesta directa o búsqueda externa (internet).

(ii) Procesamiento de contexto y recuperación (capa de inteligencia)

Una vez definida la ruta, el sistema delega la ejecución a "manejadores" o *handlers* especializados:

- Módulo RAG: si la consulta requiere datos de manuales o políticas, el VectorStoreHandler interactúa con la base de datos vectorial en Supabase. El sistema utiliza el identificador de organización (*org_id*) como filtro obligatorio para garantizar la estanqueidad y seguridad de la información. En este punto, se recuperan los fragmentos de texto (*chunks*) que presentan la mayor similitud semántica con la consulta.
- Módulo de tablas: si la información es de carácter técnico o reside en archivos estructurados (como .csv o .txt), el TablasHandler realiza una búsqueda de coincidencia exacta. Este módulo es fundamental para recuperar datos numéricos o normativos específicos que podrían perder precisión en una búsqueda puramente semántica.

(iii) Síntesis y razonamiento (capa de consolidación)

En este bloque, el contexto recuperado se reinyecta al LLM junto con la consulta original. La capacidad agéntica del sistema adquiere relevancia en esta etapa, en tanto no se limita a una simple transcripción de hallazgos. El modelo aplica una lógica de razonamiento crítico para validar si el contexto recuperado es suficiente y pertinente para responder.

Un aspecto diferencial de esta arquitectura es su mecanismo de auto-corrección o *fallback*: si el sistema determina que el módulo RAG no ha devuelto información concluyente (por ejemplo, si los *scores* de similitud son bajos), el flujo está programado para activar automáticamente el módulo de tablas mediante el comando interno [USAR_TABLA]. Esta iteración permite al agente enriquecer la respuesta con fuentes alternativas antes de dar una conclusión al usuario, lo que minimiza el riesgo de alucinaciones.

(iv) Entrega de respuesta y trazabilidad

Finalmente, la respuesta se genera de forma fluida (*streaming*) hacia la interfaz de usuario, lo que mejora la experiencia de interactividad. De forma simultánea, entra en juego el componente de telemetría a través de la clase AgenteTracer. Este módulo registra meticulosamente cada "paso" del pensamiento del agente: desde la decisión del *router* hasta la evaluación del contexto recuperado. Esta funcionalidad no solo es vital para el desarrollo técnico, sino que permite una auditoría completa del proceso de decisión, aspecto esencial para garantizar total transparencia sobre por qué el sistema eligió una fuente de datos específica o por qué activó un mecanismo de respaldo.

3.1.2. Casos de uso corporativos: aplicación práctica en banco y estudio contable

La versatilidad de la arquitectura propuesta reside en su capacidad para adaptarse a distintos contextos organizacionales mediante la configuración de parámetros específicos, sin necesidad de alterar el núcleo del sistema. A continuación, se detallan los dos casos de uso que han sido de utilidad como base para la validación de la solución.

Caso 1: entorno bancario (gestión de cumplimiento en el área de seguros)

En el sector bancario, la precisión de la información y el cumplimiento normativo son críticos. Para este caso práctico, el sistema ha sido configurado para actuar como un asistente experto en la consulta de políticas de seguros, manuales operativos y condiciones generales de pólizas.

- **Aplicación del diseño:** este escenario hace un uso intensivo del Módulo RAG. Dado que el área maneja un volumen considerable de documentos textuales (PDFs con cláusulas, exclusiones y normativas), el sistema permite a los gestores obtener respuestas inmediatas y precisas, así como también disponer de la fuente exacta de la información.
- **Valor diferencial:** aunque en esta etapa piloto el enfoque se limita a un área específica, la arquitectura garantiza la estanqueidad del conocimiento. Esto significa que el repositorio de seguros está blindado mediante su propio identificador (`org_id`), lo que permite consolidar la infraestructura necesaria para que, en futuras fases, otras divisiones del banco (como riesgos o comercial) puedan integrarse de manera aislada y segura.

Caso 2: estudio contable (gestión fiscal y datos estructurados)

El entorno de un estudio contable se caracteriza por una alta dependencia de la actualización normativa fiscal y la consulta frecuente de datos específicos o tablas de impuestos que requieren una respuesta exacta.

- **Aplicación del diseño:** a diferencia del caso bancario, este escenario requiere una integración equilibrada entre el módulo RAG y el módulo de tablas. Mientras el RAG resuelve dudas sobre leyes tributarias publicadas en boletines, el `TablasHandler` permite al sistema realizar búsquedas de coincidencia exacta en archivos estructurados (`.csv` o `.md`) que contienen calendarios fiscales o alícuotas vigentes.
- **Valor diferencial:** el flujo agéntico cobra especial relevancia al enfrentar consultas en que la semántica no es suficiente. Por ejemplo, si un contador pregunta por un coeficiente específico de amortización, el sistema puede activar

el mecanismo de *fallback* para extraer el dato exacto de una tabla técnica, y, de esa manera, evitar las imprecisiones o alucinaciones que podrían surgir de un análisis puramente textual.

3.1.3 Diseño multimodal y multilingüe: gestión de diferentes tipos de archivos y lenguajes

La eficacia de una solución de gestión del conocimiento en entornos corporativos globales depende de su capacidad para procesar información heterogénea y comunicarse en el idioma preferido por el usuario. Por ello, el sistema ha sido dotado de capacidades multimodales y multilingües que permiten una interacción fluida y sin fricciones técnicas.

Gestión multimodal de archivos

El diseño del sistema permite la ingesta y procesamiento de diversos formatos de datos, lo que garantiza que el conocimiento no quede cautivo en un solo tipo de archivo. Esta capacidad se articula a través del componente KnowledgeIndexer, encargado de transformar información no estructurada y semiestructurada en vectores procesables:

- Documentos no estructurados (PDF y Word): el sistema descompone manuales, pólizas y normativas en fragmentos (*chunks*) lógicos, a la vez que preserva la jerarquía de la información de forma que el modelo pueda referenciar secciones específicas de un documento extenso.
- Datos estructurados y semi-estructurados (.csv, .txt, .md): mediante el uso del TablasHandler, la arquitectura procesa información organizada en filas, columnas o listas. Esto es particularmente relevante en el caso del estudio contable, pues los calendarios de impuestos o las alícuotas requieren un tratamiento de datos exactos que el procesamiento de texto plano no siempre satisface.

Capacidades multilingües y transversalidad idiomática

Uno de los pilares de la propuesta es su independencia del idioma original de los datos. Por medio de la utilización de modelos de lenguaje avanzados y técnicas de

cross-lingual embeddings, el sistema opera bajo una lógica de comprensión semántica universal:

- Consultas e ingesta: el usuario puede realizar preguntas en un idioma (por ejemplo, español) sobre documentos que originalmente están redactados en otro (como inglés o catalán). El sistema identifica el significado subyacente más allá de la literalidad de las palabras, y recupera el contexto relevante sin importar el idioma de origen.
- Generación adaptativa: el componente de síntesis final está programado para responder en el idioma en el que se le ha consultado, por lo que garantiza que la entrega de la información sea natural y accesible para el empleado, independientemente de la fuente técnica consultada.

Esta dualidad en el diseño asegura que la herramienta sea una solución integral, capaz de unificar el conocimiento disperso en distintos departamentos y geografías bajo una única interfaz de consulta inteligente.

3.1.4. Flujo agéntico: implementación del razonamiento para la toma de decisiones

El componente diferencial de la solución propuesta radica en su naturaleza agéntica. A diferencia de los sistemas tradicionales de recuperación de información, que ejecutan comandos de manera lineal, este sistema integra una capa de razonamiento que le permite evaluar, decidir y rectificar sus propias acciones en tiempo real. La implementación de este flujo se desglosa en tres fases críticas:

(i) Evaluación intencional y enrutamiento (Router)

El proceso comienza con un análisis cognitivo de la consulta del usuario. A través de un componente denominado *Router*, el sistema utiliza un *prompt* de sistema altamente especializado que actúa como un primer filtro de decisión. El agente evalúa la complejidad de la pregunta y determina, de manera autónoma, qué herramienta de su inventario es la más adecuada para resolverla.

Este enrutamiento no es estático; el agente está programado para categorizar la consulta en rutas de ejecución (RAG, TABLA, DIRECTO, INTERNET), lo que permite

optimizar el uso de recursos computacionales y garantizar que el método de búsqueda se alinee con la naturaleza del dato requerido.

(ii) Ciclo de razonamiento y recuperación enriquecida

Una vez seleccionada la ruta, el agente no se limita a entregar el primer resultado encontrado. El flujo agéntico incorpora una fase de validación de contexto. Si el agente decide que la consulta es de tipo RAG, pero los datos recuperados de la base de datos vectorial no alcanzan un umbral de relevancia suficiente para garantizar una respuesta veraz, entra en juego la capacidad de decisión autónoma.

En estos escenarios, el sistema puede invocar una acción de respaldo (*fallback*). Por ejemplo, si el agente identifica que la respuesta no está en el lenguaje narrativo de los manuales, puede decidir por sí mismo realizar una búsqueda en el módulo de tablas técnicas. Este comportamiento emula el razonamiento humano: si la respuesta no se encuentra en la teoría, se busca en la documentación técnica o en los datos estructurados.

(iii) Síntesis crítica y transparencia (telemetría)

La fase final del flujo agéntico es la síntesis de la respuesta bajo criterios de suficiencia. Una vez que el agente recibe el contexto recuperado (ya sea de vectores o tablas), realiza un juicio de valor técnico antes de generar la respuesta definitiva. En lugar de limitarse a una transcripción literal de los fragmentos encontrados, el LLM evalúa si la información es suficiente para satisfacer la consulta de manera íntegra y veraz.

Si el agente determina que el contexto es parcial o contradictorio, tiene la capacidad de contextualizar la respuesta e indicar las limitaciones de lo hallado o, en última instancia, activa el mecanismo de *fallback* para buscar en una fuente alternativa. Este proceso garantiza una "respuesta consolidada", donde el sistema no solo entrega datos, sino que los organiza y valida según la necesidad específica del usuario, lo que permite reducir drásticamente el riesgo de alucinaciones o respuestas fuera de contexto.

Para garantizar la fiabilidad de este proceso, cada una de estas decisiones es registrada por el componente de telemetría. Esto permite que el usuario o el administrador del

sistema puedan visualizar el "camino de pensamiento" que siguió el agente. La transparencia en la toma de decisiones agénticas es fundamental para generar confianza en entornos corporativos, pues posibilita auditar por qué el sistema priorizó una fuente de datos sobre otra o por qué decidió cambiar de estrategia de búsqueda durante la ejecución.

3.2. Implementación técnica

La implementación del presente trabajo se construye sobre una arquitectura de software moderna que integra tecnologías especializadas en procesamiento de lenguaje natural, almacenamiento vectorial y desarrollo de aplicaciones web escalables. Esta sección describe de manera integral los componentes tecnológicos seleccionados y justifica su utilización en el contexto específico de una plataforma RAG multimodal multi-tenant.

3.2 Stack tecnológico

4.1.1 Componentes del Backend

Backend de procesamiento de lenguaje natural

El procesamiento de consultas y generación de respuestas se fundamenta en FastAPI (versión $\geq 0.110.0$), un framework asincrónico de alto rendimiento para construcción de APIs REST. FastAPI ha sido seleccionado por su capacidad de manejar operaciones no-bloqueantes, lo que resulta especialmente ventajoso en contextos donde múltiples clientes envían consultas simultáneamente. Adicionalmente, proporciona documentación automática mediante Swagger y OpenAPI, facilitando la integración con frontends diversos.

Para la ejecución del servidor ASGI, se utiliza Uvicorn (versión $\geq 0.28.0$), un servidor HTTP asincrónico que permite la concurrencia eficiente de peticiones. Esta combinación FastAPI + Uvicorn permite desplegar la aplicación en plataformas serverless como Vercel, reduciendo costos de infraestructura.

Orquestación de Modelos de Lenguaje

El ecosistema de LangChain proporciona abstracciones de alto nivel para interactuar con múltiples proveedores de modelos de lenguaje, evitando el acoplamiento a un único proveedor. En el presente trabajo se integran:

langchain-google-genai (versión $\geq 2.0.0$): Proporciona acceso a Google Generative AI (Gemini 2.0 Flash) en modo cloud.

langchain-ollama (versión $\geq 0.3.0$): Permite ejecutar modelos locales de código abierto (Phi-3, Llama 3) mediante Ollama, viable cuando se dispone de infraestructura GPU local.

langchain-core (versión $\geq 0.1.0$): Define primitivas abstractas para cadenas de procesamiento.

langchain-community (versión $\geq 0.4.1$): Proporciona integradores con herramientas y servicios externos.

langchain-text-splitters (versión $\geq 1.1.1$): Implementa estrategias de fragmentación de documentos para optimizar la ingesta de conocimiento.

Esta arquitectura permite que el sistema funcione de manera agnóstica respecto al backend de IA subyacente, permitiendo migración entre proveedores sin cambios significativos en la lógica de negocio.

Modelos de Embeddings

La transformación de texto en representaciones vectoriales se realiza mediante embeddings semánticos. Se han integrado dos enfoques:

Google Generative AI Embeddings (modelo gemini-embedding-2): Embeddings con dimensionalidad configurable (por defecto 1536), optimizados para consultas en cloud.

Ollama Embeddings: Para inferencia local de modelos como all-MiniLM-L6-v2, cuando se ejecuta en modo local.

La selección del modelo de embeddings afecta directamente la calidad de recuperación en la fase de retrieval del pipeline RAG, por lo que se ha hecho configurable por cliente.

Acceso a APIs Externas

OpenAI (versión $\geq 2.31.0$): Proporciona integración alternativa con modelos GPT-4, aunque no se ha utilizado como backend principal en esta fase.

supabase (versión $\geq 2.30.0$): Cliente Python para acceso a PostgreSQL con extensiones vectoriales y gestión de usuarios.

4.1.2 Componentes de Almacenamiento y Persistencia

Base de Datos Vectorial

Chroma (integrado mediante langchain-chroma $\geq 1.1.0$) se utiliza como motor de almacenamiento vectorial persistente. Chroma ofrece persistencia en disco mediante SQLite3, soporte para múltiples funciones de similitud (cosine, euclidean, dot product), escalabilidad a millones de vectores y una API simple para CRUD de documentos. La base vectorial se persiste en la carpeta db/vector_store/ dentro de cada tenant, permitiendo reconstrucción eficiente sin re-procesar documentos.

Base de Datos Relacional

Supabase (PostgreSQL + PgVector) almacena información de usuarios y perfiles, históricos de consultas para auditoría, configuraciones específicas por cliente y aprovecha la extensión pgvector para búsquedas vectoriales de baja latencia.

Procesamiento de Documentos

PyPDF (versión $\geq 6.9.2$): Extracción de texto desde archivos PDF, con manejo robusto de codificaciones.

openpyxl (versión $\geq 3.1.5$): Lectura de hojas de cálculo Excel para ingesta de tablas de referencia.

4.1.3 Componentes de Análisis de Datos

Pandas (versión $\geq 3.0.1$): Manipulación y análisis de datos tabulares, especialmente para procesamiento de CSVs y tablas de datos estructurados.

tabulate (versión $\geq 0.9.0$): Formateo de tablas para presentation legible en logs y reportes de telemetría.

plotly (versión $\geq 6.7.0$): Visualización interactiva de métricas y gráficos de análisis.

matplotlib (versión $\geq 3.10.9$): Generación de gráficos estáticos para reportes.

4.1.4 Frontend: Next.js 15 con React

El frontend SaaS se construye sobre Next.js 15, un framework meta-fullstack que proporciona App Router (sistema de enrutamiento basado en directorios con soporte nativo para layouts jerárquicos), Server Components (reducción de código JavaScript

enviado al cliente mediante renderización en servidor), Streaming SSR (respuestas progresivas para mejor UX) y optimización automática (imágenes, fuentes, code splitting).

Se complementa con:

React: Librería de UI con hooks y context API para gestión de estado.

React-DOM: Renderización en el navegador del lado del cliente donde sea necesario.

TypeScript (versión $\geq 5.6.3$): Tipado estático para reducción de errores en desarrollo.

4.1.5 Estilos y Diseño

Tailwind CSS (versión $\geq 3.4.15$): Framework de utilidades CSS para diseño responsivo sin escribir CSS custom. Proporciona un sistema de diseño consistente.

PostCSS (versión $\geq 8.4.49$): Procesador de CSS que integra autoprefixer y transformaciones de Tailwind.

Framer Motion (versión $\geq 11.11.17$): Librería de animaciones declarativas para transiciones y micro-interacciones.

Lucide React (versión $\geq 0.460.0$): Sistema de iconos SVG consistente.

4.1.6 Autenticación y Autorización

Clerk (versión $\geq 6.3.3$): Plataforma de autenticación y gestión de usuarios con soporte integrado para múltiples métodos de login (email, OAuth con Google, GitHub, Microsoft), gestión de roles y permisos (RBAC), webhooks para eventos de usuario y SSO empresarial.

@supabase/supabase-js (versión $\geq 2.46.1$): Cliente JavaScript para acceso seguro a base de datos con Row Level Security (RLS).

4.1.7 Utilidades de Configuración

python-dotenv (versión $\geq 1.0.0$): Carga de variables de entorno desde archivos .env para desarrollo local.

dotenv (lado Python): Gestión centralizada de credenciales sin exponerlas en control de versiones.

4.1.8 Testing y Calidad de Código

ESLint (versión $\geq 9.14.0$) & @eslint/eslintrc (versión $\geq 3.0.0$): Linter para JavaScript/TypeScript con configuración Next.js extendible.

pytest: Framework de testing para código Python implícito en pyproject.toml.

4.1.9 Build y Deployment

npm/yarn: Gestores de dependencias para ecosistema Node.js.

Vercel: Plataforma de despliegue serverless optimizada para Next.js, con CI/CD automático.

Docker: Orquestación de infraestructura local (disponible en docker-infra/ para desarrollo).

4.1.10 Síntesis del Stack Tecnológico

La arquitectura integrada conforma un sistema moderno que combina: en la capa de presentación, tecnologías web contemporáneas que permiten experiencias de usuario responsivas e interactivas; en la capa de aplicación, frameworks especializados que abstraen la complejidad de la infraestructura; en la capa de inteligencia artificial, múltiples opciones de backends que garantizan flexibilidad operativa; y en la capa de datos, almacenamiento tanto relacional como vectorial, optimizando cada tipo de consulta para su caso de uso específico.

La selección conjunta de estos componentes responde a criterios de madurez (librería estable, ampliamente adoptada), soporte comunitario (documentación abundante, respuestas en Stack Overflow), y compatibilidad (integración sin fricciones entre componentes).

4.2 Desarrollo del Core: Clase BaseModel y Adaptaciones Específicas

4.2.1 Concepto Arquitectónico de BaseModel

La clase BaseModel constituye el núcleo computacional del sistema TFM. Su propósito fundamental es encapsular toda la lógica de orquestación de inteligencia artificial, gestión de conocimiento y procesamiento de consultas en una abstracción reutilizable y configurable. A diferencia de sistemas monolíticos, BaseModel se diseña como una clase base abstracta que permite la especialización para clientes específicos mediante herencia.

4.2.2 Principios de Diseño

Separación de Responsabilidades: Cada método tiene una responsabilidad claramente definida. No es una 'god class' sino una orquestadora que delega a especializaciones (handlers, indexers). Esta separación facilita testing unitario y comprensión del flujo.

Configurabilidad Dinámica: Toda la configuración se externaliza a archivos JSON, permitiendo cambios sin recompilación de código. Esto es crítico en una plataforma multi-tenant donde cada cliente tiene necesidades distintas. Un cambio de prompt o de modelo de embeddings requiere únicamente editar un archivo JSON.

Agnóstico de Proveedor: El sistema puede funcionar con Ollama (local) o Google Gemini (cloud) sin cambios en la lógica central. Las importaciones condicionales detectan qué librerías están disponibles, permitiendo instalación selectiva de dependencias.

Patrón RAG Explícito: Implementa explícitamente el pipeline Retrieval-Augmented Generation sin depender de frameworks de agentes complejos (como AgentExecutor de LangChain), lo que permite control fino y debugging granular.

4.2.3 Inicialización y Construcción

El ciclo de vida de una instancia de BaseModel sigue esta secuencia ordenada: En primer lugar, el constructor recibe path_cliente, una ruta hacia la carpeta del cliente. Desde esta localización, se cargan archivos de configuración: settings.json contiene configuración técnica (modelo LLM, URL de Ollama, dimensionalidad de embeddings), rag.json contiene el manifiesto del conocimiento (directorios de documentos, personalidad del agente), y ejemplos_qa.json proporciona ejemplos para few-shot prompting.

Seguidamente, mediante lógica condicional, se detecta si existe configuración para modo local (Ollama) o si debe usarse cloud (Google Gemini). Esta detección es crucial porque determina qué backends de inferencia se cargarán. A continuación, se inicializan los componentes de IA: es decir, se carga el LLM apropiado, el modelo de embeddings y se crea un trazador de telemetría para auditoría. Finalmente, se reservan recursos dejando pendiente la carga de la base vectorial (vector_store) hasta que se invoque

explícitamente `configurar_conocimiento()`. Esta estrategia de lazy initialization optimiza la memoria en contextos serverless.

4.2.4 Método `config_plano()`: Unificación de Configuraciones

Un reto importante en sistemas heredados es la compatibilidad hacia atrás. `config_plano()` es un método estático que implementa una estrategia de normalización de configuraciones potencialmente anidadas. Dado un diccionario de configuración que puede tener diferentes estructuras (anidadas o planas), `config_plano()` lo convierte en un diccionario plano con convenciones predefinidas. Por ejemplo, `cliente.org_id` se normaliza a `org_id`, `LLM.modelo` a `modelo`, y `embeddings.modelo` a `modelo_embeddings`. Esta normalización permite que el resto del código trabaje con una vista consistente, sin importar si la configuración proviene de un esquema nuevo (anidado) o de un formato heredado (plano).

4.2.5 Adaptabilidad de Backend: Ollama vs. Google Gemini

Una característica central es la capacidad de cambiar de backend sin cambiar código. Esto se logra mediante varios mecanismos:

Imports Condicionales: Al inicio del módulo, se intenta importar librerías específicas. Si no están instaladas, se marca un flag `_GOOGLE_AVAILABLE` o `_OLLAMA_AVAILABLE`. Esto permite que desarrolladores instalen solo las dependencias que necesitan.

Método `_es_local()`: Examina si existe una URL de LLM configurada. Si la hay, se asume modo local (Ollama); si no, se asume cloud (Google).

Inicialización Polimórfica: En `_inicializar_llm()`, según el modo detectado, se retorna una instancia de `ChatOllama` o `ChatGoogleGenerativeAI`. Ambas heredan de la interfaz `LangChain BaseLLM`, garantizando la compatibilidad de contrato.

De manera análoga, `_inicializar_embeddings()` retorna `OllamaEmbeddings` o `GoogleGenerativeAIEmbeddings` según el contexto. Las ventajas de este diseño son múltiples: permite desarrollo local con modelos pequeños (Phi-3) sin costos de API, facilita el escalado a producción con modelos más potentes (Gemini), permite investigación mediante estudios de ablación y evita el lock-in con un proveedor específico.

4.2.6 Gestión de Conocimiento: configurar_conocimiento()

Este método es el director de orquestación de RAG. Su lógica central dicta que, primero, evalúa si existe ya una base vectorial construida en `db/vector_store/`. Segundo, toma una decisión: si no existe o se fuerza un rebuild, invoca el `KnowledgeIndexer` para procesamiento pesado (lectura de PDFs, fragmentación, vectorización); si existe, solo carga la instancia de Chroma desde el disco (operación rápida). Tercero, una vez la base vectorial está lista, establece el prompt del sistema dinámicamente basado en la configuración del cliente. Cuarto, prepara los handlers especializados (RAG handler, handler de tablas, etc.) que usarán la base vectorial.

Las implicaciones de diseño son significativas: la primera invocación es computacionalmente costosa, pero las subsecuentes son instantáneas. En contextos serverless como Vercel, la base vectorial se carga en memoria una sola vez por instancia y se reutiliza, minimizando cold starts. Cada cliente tiene su propia carpeta y base vectorial independiente, evitando la contaminación de datos entre tenants.

4.2.7 Procesamiento Multimodal de Documentos

El sistema soporta la ingesta de múltiples tipos de contenido, cada uno con estrategias optimizadas:

PDFs: Mediante `PyPDF`, se extrae texto de todas las páginas. Se aplica `RecursiveCharacterTextSplitter` con configuración específica: chunks de 1000 caracteres con overlap de 150 caracteres. El overlap asegura que conceptos que cruzan límites de chunks no se pierdan. Cada chunk recibe metadata indicando archivo origen y módulo del conocimiento.

Tablas (CSV, TXT, MD): Se cargan pero no se vectorizan. En su lugar, se invocan mediante handlers específicos que realizan búsqueda lexical (búsqueda de string simple). Esta decisión es fundamental porque los datos tabulares son exactos y la búsqueda semántica puede introducir errores por aproximación. Una búsqueda exacta es preferible.

Contenido Web: Se ingiere como texto plano. No se aplica splitting adicional, asumiendo que el contenido ya está bien estructurado.

Imágenes: Actualmente no soportadas en producción, pero el código incluye un método stub `_procesar_imagenes()` para futura integración con modelos multimodales (Llama Vision, Gemini 2.0 Flash nativo multimodal).

4.2.8 Establecimiento del Prompt Dinámico

El método `_establecer_prompt_dinamico()` construye la estructura de diálogo que guía al LLM. El prompt se divide en capas: La capa de System Message contiene instrucciones maestras, estilo requerido y reglas de oro específicas del cliente. La capa History Placeholder permite incluir el contexto conversacional para diálogos multi-turno. La capa Template proporciona el patrón para cada turno donde se inserta el conocimiento RAG recuperado y la pregunta del usuario.

Crucialmente, el prompt incluye una instrucción de tooling: si el LLM necesita acceder a datos de tablas, debe escribir `[USAR_TABLA: término_de_búsqueda]`. Esta sintaxis fácilmente detectable mediante expresiones regulares (regex) permite al LLM 'invocar herramientas' sin frameworks complejos de agentes.

4.2.9 Enrutamiento Inteligente de Consultas

El método `_evaluar_intencion()` implementa un 'router' que clasifica cada pregunta. Este router es en sí un LLM prompt específicamente diseñado que solicita al modelo evaluar cinco dimensiones y responder con una palabra clave. Las categorías de intención son:

DIRECTO: Cuando la pregunta es sobre conocimiento general que el LLM posee inherentemente sin contexto externo.

RAG: Cuando se necesita recuperar información de los documentos vectorizados.

TABLA: Cuando se requieren datos exactos de tablas de referencia o bases de datos.

INTERNET: Cuando se necesita información actual/reciente no disponible localmente.

OTRO: Cuando la pregunta está fuera del scope o no puede ser respondida.

Este enrutamiento explícito es superior a un sistema 'siempre-RAG' porque evita contexto innecesario cuando no es requerido, permite la especialización y mejora la interpretabilidad del sistema (se sabe exactamente qué subsistema manejó la pregunta).

4.2.10 Pipeline de Respuesta: Patrón RAG React

El método `responder()` implementa un pipeline de cuatro fases que generaliza el patrón RAG:

Fase 1 - Recuperación y Filtrado: Se vectoriza la pregunta del usuario, se buscan los $k=10$ documentos más similares, se aplica un umbral de similitud donde solo documentos con $\text{score} < 0.85$ se consideran relevantes, y documentos con $\text{score} > 0.85$ se descartan como ruido.

Fase 2 - Generación Inicial: Se ensambla el prompt con el contexto RAG filtrado y la pregunta, se invoca el LLM y se obtiene la respuesta inicial.

Fase 3 - Detección de Tooling: Se aplica regex para detectar si la respuesta contiene `[USAR_TABLA: ...]` y se extrae el término de búsqueda si es necesario.

Fase 4 - Re-intento con Datos de Tablas (si aplica): Se invoca `consultar_tablas_y_db(término)` que hace búsqueda lexical, se enriquece el contexto RAG con los datos encontrados, se vuelve a invocar el LLM con contexto aumentado y se retorna la respuesta mejorada.

El método usa `yield` (generators de Python) en lugar de retornar una respuesta completa. Esto permite el streaming de respuestas al cliente: conforme el LLM genera tokens, estos se envían progresivamente al navegador, mejorando la UX percibida significativamente.

4.2.11 Herramientas Virtuales: Consulta de Tablas

El método `consultar_tablas_y_db()` implementa una herramienta simulada de acceso a bases de datos. Cuando el LLM detecta que necesita información exacta, invoca esta herramienta escribiendo `[USAR_TABLA: ...]`. La implementación es un 'crawler' lexical que itera sobre todos los módulos del conocimiento de tipo 'tablas', lee cada archivo línea por línea, busca la ocurrencia del término con búsqueda case-insensitive y retorna un 'buffer' alrededor de la coincidencia para contexto. Esto es más eficiente que vectorizar tablas porque los datos exactos se recuperan exactamente, evitando pérdida de precisión por cuantización de embeddings.

4.2.12 Adaptaciones por Cliente: Herencia y Especialización

Cada cliente específico (e.g., banco, estudio contable) hereda de `BaseModel` y solo necesita especificar la ruta hacia su carpeta de configuración, invocando luego

`configurar_conocimiento()`. Toda la configuración específica reside en archivos dentro de la carpeta del cliente: qué LLM usar, qué modelos de embedding, dónde están los documentos y cuál es la personalidad del agente. Esto permite agregar nuevos clientes sin tocar código Python, simplemente creando una carpeta con estructura estándar y un archivo JSON de configuración.

4.2.13 Telemetría y Debugging

La clase incluye un sistema de trazabilidad que registra cada paso: pregunta recibida, intención clasificada, documentos recuperados y sus scores, respuesta cruda del LLM antes de post-procesamiento, intento de acceder a tablas y respuesta final. Estos logs se guardan en `evaluaciones_pendientes.json` para posterior análisis. Permite identificar queries que el router clasifica incorrectamente, documentos con embeddings de baja calidad y casos donde el sistema falla y necesita intervención humana.

4.3 Interfaz y Plataforma SaaS: Desarrollo del Entorno de Usuario

4.3.1 Visión Arquitectónica del Frontend

La interfaz de usuario de TFM se construye como una plataforma SaaS moderna, donde múltiples clientes (tenants) acceden a través de un portal web unificado con experiencias personalizadas. El frontend es responsable de gestionar autenticación, proporcionar la interfaz conversacional, permitir la administración de documentos, visualizar analíticas y facilitar la configuración por cliente.

4.3.2 Selección de Next.js 15 como Framework

Next.js 15 ha sido seleccionado por varias razones estratégicas fundamentales para el desarrollo de aplicaciones web modernas. Su capacidad full-stack permite un único repositorio que aloja tanto frontend como backend API, simplificando el despliegue y la sincronización. La arquitectura de App Router proporciona una forma más intuitiva de estructurar aplicaciones comparado con enfoques anteriores, permitiendo layouts jerárquicos reutilizables, route groups para organización conceptual y soporte nativo para server y client components.

Los Server Components permiten componentes React renderizados en el servidor, reduciendo el JavaScript enviado al cliente (lo que mejora el rendimiento), habilitando el acceso seguro a secretos desde el servidor y permitiendo consultas a la base de datos

sin exponer credenciales. El Streaming SSR permite enviar HTML progresivamente, mejorando el Time to First Contentful Paint (FCP) significativamente. Asimismo, las optimizaciones automáticas de Next.js optimizan de forma nativa imágenes, fuentes y code splitting, reduciendo la carga manual de optimización. Finalmente, la integración con Vercel proporciona un despliegue optimizado con zero-config, CI/CD automático y rollbacks instantáneos.

4.3.3 Estrategia de Autenticación Multinivel

La autenticación implementa un esquema de defensa en profundidad mediante múltiples capas:

Capa 1 - Clerk (Cliente HTTP): Proporciona la interfaz de login/signup. Clerk es un proveedor de identidad especializado que maneja múltiples métodos de login (email, OAuth), autenticación de dos factores, gestión de sesiones y rate limiting contra ataques de fuerza bruta.

Capa 2 - Middleware de Next.js: Intercepta todas las peticiones y verifica que el token JWT de Clerk sea válido. Si el usuario accede a una ruta protegida sin un token válido, es redirigido automáticamente a la página de login.

Capa 3 - Row Level Security (RLS) en Supabase: A nivel de base de datos, las políticas de RLS aseguran que incluso si un atacante obtuviera acceso a las APIs de Supabase, no podría ver datos de otros usuarios. Las consultas se filtran automáticamente usando `auth.uid()`.

Capa 4 - Secretos Serverless: Las claves de API para Gemini, tokens de Supabase, etc., se almacenan de forma segura en las variables de entorno de Vercel (no en el código fuente) y nunca se exponen al cliente.

4.3.4 Componentes de Experiencia de Usuario

Componente Chat Conversacional: Proporciona la interfaz conversacional principal con características como historial persistente, indicador de estado ('pensando'), soporte Markdown en respuestas, fuentes citadas cuando RAG recupera documentos y scroll automático hacia el mensaje más reciente.

Componente Perfil de Usuario: Permite a los usuarios ver información de su cuenta, actualizar preferencias personales, generar claves de API para integración con herramientas externas y visualizar su historial de uso.

Componente Gestor de Conocimiento: Disponible exclusivamente para administradores de tenants, permite el upload de documentos, vista previa de documentos indexados, trigger manual de re-indexación y estadísticas de cobertura.

4.3.5 Backend API Serverless: FastAPI en Vercel

El backend se despliega en Vercel como una aplicación serverless, lo que significa ausencia de servidores que gestionar, escalado automático según la carga, facturación por uso exacto y cold starts inherentes a la arquitectura (con una primera invocación de 1-2 segundos). Los endpoints clave incluyen:

`/api/health`: Proporciona diagnóstico detallado: disponibilidad de LLM (Ollama vs. Gemini), variables de entorno cargadas correctamente, estado de conectividad con la base de datos vectorial y versiones de modelos activos. Es crítico para el debugging remoto.

`/api/chat`: Procesa consultas aceptando el mensaje del usuario, historial conversacional (para contexto multi-turno) e ID de la organización (para multi-tenancy). Retorna el contenido de la respuesta, fuentes/documentos utilizados y metadatos de rendimiento.

`/api/embeddings/recompute`: Permite al administrador de un tenant forzar la re-indexación de documentos, útil después de actualizar archivos, cambiar el modelo de embeddings o debuggear problemas de recuperación.

4.3.6 Gestión de Estado Global con React Context

Para evitar el prop drilling (pasar props a través de múltiples niveles innecesariamente), se implementan contextos React:

`AuthContext`: Proporciona acceso al usuario actual, información de rol y métodos de login/logout en cualquier componente de la aplicación.

`ChatContext`: Mantiene el historial de mensajes del chat activo, el estado de carga y los métodos para enviar nuevos mensajes en tiempo real.

KnowledgeContext: Proporciona a los administradores de la organización acceso a los metadatos de documentos indexados y métodos de subida de archivos.

4.3.7 Integración con Supabase para Persistencia

Supabase sirve como backend de datos relacionales mediante un conjunto estructurado de tablas: la tabla 'users' (perfiles vinculados a Clerk IDs), la tabla 'organizations' (tenants/clientes), la tabla 'conversations' (historiales de chat), la tabla 'documents' (metadatos de documentos indexados) y la tabla 'audit_logs' (trazabilidad de acciones de administración). Las consultas se realizan a través del cliente oficial de Supabase en JavaScript, que maneja encriptación en tránsito (HTTPS), Row Level Security y reconexión automática ante errores de red.

4.3.8 Flujo Completo de Usuario

El flujo típico de un usuario comienza cuando accede a la plataforma en su navegador. Si no está autenticado, es redirigido a Clerk. Tras un login exitoso, Clerk emite un JWT que se almacena en una cookie segura. El middleware de Next.js verifica este JWT, permitiendo el acceso. Se carga el componente Chat, recuperando el historial de Supabase. Cuando el usuario escribe un mensaje y presiona Enter, el frontend hace una petición POST a /api/chat. El backend BaseModel procesa la consulta (evaluando la intención y recurriendo a RAG o tablas según aplique). La API retorna la respuesta en tiempo real usando streaming. El frontend renderiza la respuesta progresivamente con formato Markdown y referencias. Finalmente, Supabase persiste el intercambio completo para futuras referencias en el hilo.

4.3.9 Consideraciones de Performance

Code Splitting: Next.js divide automáticamente el código en pequeños chunks, asegurando que solo se descargue el código necesario para la página actual.

Image Optimization: Las imágenes se convierten de forma nativa a formatos modernos (como WebP) y se sirven en múltiples resoluciones adaptadas al dispositivo.

Caching Estratégico: Los componentes estáticos se cachean en una CDN global, los datos dinámicos se mantienen en memoria del cliente durante la sesión y las respuestas de la API utilizan estrategias de stale-while-revalidate.

Lazy Loading: Los componentes e interfaces pesadas (como gráficos de analíticas masivos) se cargan de forma diferida y bajo demanda.

4.4 Gestión de Configuraciones

4.4.1 Problema: Multi-Tenancy y Heterogeneidad de Clientes

Un desafío central en sistemas SaaS es la heterogeneidad: cada cliente tiene diferentes modelos de IA preferidos, diferentes documentos de referencia, diferentes estilos de respuesta y diferentes roles de usuario. La solución implementada es una arquitectura de configuración externalizada donde la lógica de la aplicación es completamente agnóstica, toda la customización reside en archivos de configuración estructurados y no es necesario recompilar o redispengar la plataforma para cambiar el comportamiento de un cliente.

4.4.2 Estructura de Carpetas Multi-Cliente

Dentro del directorio principal RAG-docs/, cada cliente posee su propia carpeta aislada con una estructura estandarizada: config/ aloja los archivos de configuración JSON; archivos/ contiene los documentos organizados por tipo (pdfs, tablas, web); db/ contiene la base vectorial persistente Chroma; y archivos de tracking como evaluaciones_pendientes.json contienen los logs analíticos. Esta estructura permite dar de alta nuevos clientes de forma inmediata simplemente creando una carpeta con la estructura estándar, sin alterar el código fuente de la aplicación.

4.4.3 Schema de settings.json (Configuración Moderna)

El archivo settings.json representa el declarativo moderno de configuración de un cliente. Su estructura interna incluye de forma jerárquica una sección 'cliente' con metadatos del tenant (ID único, nombre para la UI, personalidad), una sección 'LLM' con la selección del modelo y parámetros de inferencia (temperatura, top_p, max_tokens), una sección 'embeddings' con la configuración del modelo de vectorización, una sección 'indice_conocimiento' con el mapeo de módulos a directorios físicos y una sección 'instrucciones_sistema' con el prompt base y la personalidad requerida.

4.4.4 Backward Compatibility: rag.json (Formato Legacy)

Para garantizar la compatibilidad hacia atrás con sistemas preexistentes, el formato `legacy rag.json` sigue estando soportado. La lógica de carga implementada sigue una estrategia resiliente: el sistema intenta cargar en primera instancia `settings.json` y verifica si contiene las secciones críticas; si estas no se encuentran o el archivo está ausente, lee automáticamente `rag.json` y fusiona dinámicamente sus campos con el esquema moderno de `settings.json`. Esto permite una migración gradual y segura de clientes sin interrumpir el servicio.

4.4.5 Gestión de Secretos: Variables de Entornos

Las credenciales sensibles y las claves de API nunca se incluyen en el control de versiones. En desarrollo local, se utilizan archivos `.env.local` (estrictamente incluidos en `.gitignore`). En producción, las mismas variables se definen directamente en la consola de Vercel como 'Secrets' cifrados, invisibles en los logs y en los procesos de compilación. La carga segura en Python implementa una lógica defensiva: carga `.env.local` si está presente en entornos locales, lee las variables mediante `os.getenv()` y valida de forma estricta la presencia de secretos obligatorios según el modo detectado (local cloud), levantando excepciones tempranas ante la falta de credenciales.

4.4.6 Configuración de Frontend: Archivos de Build

`next.config.ts`: Personaliza el comportamiento de compilación y empaquetado de Next.js, permitiendo optimizaciones avanzadas de compilación y el manejo pragmático de warnings de linters durante fases de desarrollo.

`tailwind.config.ts`: Define el sistema de diseño visual de la interfaz (paleta de colores corporativos, tipografía global, breakpoints responsivos), permitiendo alterar la estética de la plataforma de manera centralizada sin modificar componentes de UI individuales.

`tsconfig.json`: Configura el compilador de TypeScript para asegurar un tipado estricto, la detección temprana de valores null/undefined y garantizar la compatibilidad óptima de módulos de software.

4.4.7 Configuración de Despliegue: `vercel.json`

El archivo `vercel.json` especifica el comportamiento exacto del despliegue en la nube: define los comandos de compilación (build commands), el directorio de salida (output

directory), las reglas de enrutamiento y reescritura de URLs para la API asincrónica de FastAPI y el mapeo de variables de entorno hacia los secrets de la infraestructura.

4.4.8 Configuración de Linting: eslint.config.mjs

Define las reglas estáticas de análisis de código para mantener la calidad y consistencia del software en todo el equipo de desarrollo. Especifica los plugins de Next.js y React activos, y establece reglas estrictas para prevenir malas prácticas en la renderización HTML o la gestión de dependencias.

4.4.9 Estrategia de Caché y Persistencia

Para maximizar la eficiencia y reducir los costes operativos, el sistema implementa una estrategia de persistencia multinivel:

Caché en Memoria (Backend): En la instancia de FastAPI se mantiene un diccionario global de objetos cliente instanciados. La primera solicitud entrante es computacionalmente costosa (~5-10 segundos por la carga e inicialización de BaseModel), pero todas las llamadas subsecuentes se resuelven en menos de 1ms al reutilizar la instancia caliente en memoria.

Persistencia de Embeddings (Disco): La base de datos vectorial Chroma se almacena físicamente en el disco de la carpeta db/ vector_store/, lo que permite levantar el contexto y realizar búsquedas de similitud de manera inmediata sin necesidad de consumir APIs externas para volver a vectorizar documentos históricos.

Logs de Auditoría (JSON): Cada interacción, clasificación de intención y métricas de rendimiento se vuelcan en tiempo real en archivos estructurados JSON, permitiendo auditorías offline y análisis de precisión sintáctica sin penalizar la latencia de la base de datos relacional principal.

Evaluación de modelos de *embeddings* y rendimiento

Este capítulo presenta el análisis comparativo de los diferentes modelos de *embeddings* evaluados para la solución. El objetivo es determinar la configuración que optimice la recuperación de información, equilibrando la precisión semántica con la eficiencia en el consumo de recursos.

5.1 Diseño Experimental: Selección de modelos y configuración de pruebas

Para garantizar la objetividad de los resultados, se diseñó un entorno de pruebas controlado utilizando el script `embedding_test.py`. El experimento se estructuró de la siguiente manera:

- Selección de Modelos: Se evaluaron diversos modelos de la librería HuggingFace, centrando el análisis en modelos multilingües capaces de procesar la terminología específica del sector bancario y contable (ej. `intfloat/multilingual-e5-small`).
- Configuración de Variables: Se ejecutó un bucle de pruebas iterando sobre tres variables críticas: el modelo de *embedding*, el tamaño del fragmento (Chunk Size) y el solapamiento (Overlap).
- Ground Truth: Se generó un conjunto de datos de referencia (preguntas de prueba) basado en el texto real de los PDFs cargados, lo que permitió comparar la respuesta del sistema contra una "verdad conocida".

5.2 Métricas de Precisión: Análisis de MRR y Top-K Accuracy

La calidad de la recuperación de información se midió mediante métricas estándar de *Information Retrieval*:

- MRR (Mean Reciprocal Rank): Evalúa qué tan "arriba" en la lista de resultados aparece el fragmento correcto. Los resultados destacaron que configuraciones equilibradas logran un MRR de 0.85, lo que indica que el sistema encuentra la información correcta casi siempre en la primera opción.
- Top-K Accuracy (Top-1 y Top-3): Se midió la probabilidad de que la respuesta correcta estuviera entre los primeros resultados. El análisis mostró un Top-3 cercano al 89%, validando la robustez del motor de búsqueda semántica.

5.3 Evaluación Cualitativa: Relevancia de las respuestas obtenidas

Más allá de las métricas numéricas, se implementó un LLM Judge (Evaluador basado en LLM). Este componente analiza la calidad de la síntesis final:

- Se evaluó si el fragmento recuperado realmente respondía a la pregunta del usuario o si era solo semánticamente similar pero irrelevante.

- Los resultados cualitativos confirmaron que la combinación de búsqueda semántica y el razonamiento del agente reduce las alucinaciones, entregando respuestas que no solo son precisas, sino que mantienen el contexto técnico necesario para el área de seguros y contabilidad.

5.4 Impacto de los Parámetros: Relación entre Chunk Size, Overlap y rendimiento

Uno de los hallazgos más importantes del estudio es la relación entre la segmentación del texto y el rendimiento del sistema:

- **Chunk Size (Tamaño de fragmento):** Se compararon tamaños de 500 y 1000. Los datos revelaron que un Chunk Size de 1000 es óptimo. Al usar fragmentos de tamaño medio, se reduce el número de bloques a procesar a la mitad (bajando de 49 a 23 en las pruebas), lo que disminuye drásticamente el consumo de tokens (de 4240 a 2222) sin sacrificar la precisión.
- **Overlap (Solapamiento):** Se determinó que solapamientos excesivos (150/300) saturan la base de datos y confunden al motor de similitud. Un Overlap mínimo de seguridad (50) demostró ser suficiente para no perder el contexto entre fragmentos.
- **Conclusión de Rendimiento:** El modelo **multilingual-e5-small** con la configuración 1000/50 se identificó como la opción "todoterreno", logrando un uso de RAM estable (2318 MB) y tiempos de procesamiento ágiles, siendo la decisión arquitectónica más sólida para la implementación final.

2. Resultados

El TFM ha producido una plataforma inteligente de RAG (Retrieval-Augmented Generation) completamente funcional con resultados demostrables en múltiples dimensiones:

1. Arquitectura y Sistema Core Implementado

- **Motor de Inteligencia Artificial Modular:** Se desarrolló la clase `BaseModel` en `base_llm.py`, que orquesta un sistema híbrido capaz de enrutar automáticamente cada consulta hacia el handler más apropiado (RAG, tablas estructuradas, búsquedas en internet o respuesta directa).
- **Base de Conocimiento Vectorial:** Se indexaron correctamente 44 fragmentos (chunks) de documentos de productos de seguros utilizando Supabase como vector store, logrando una organización escalable por organización (`org_id`).
- **Reducción de Alucinaciones:** Se implementó un umbral de defensa contra alucinaciones ($\text{score} < 0.85$) en el proceso de recuperación semántica, filtrando documentos que no tienen suficiente similitud con la consulta del usuario.

2. Evaluación de Modelos de Embedding

Se realizó un análisis exhaustivo multi-métrica comparando 7 modelos de embedding diferentes:

- **Modelos evaluados:** `paraphrase-multilingual-MiniLM-L12-v2`, `multilingual-e5-small`, `multilingual-e5-base`, `distiluse-base-multilingual-cased-v1`, `jina-embeddings-v3`, `jina-embeddings-v2-base-es`, y `LaBSE`.
- **Configuraciones testadas:** 3 tamaños de chunk (500, 1000, 1500) $\times$ 3 valores de overlap (50, 150, 300).
- **Métricas implementadas:**
 - **Mean Reciprocal Rank (MRR):** Evalúa la posición del documento correcto en el ranking de recuperación.
 - **Accuracy Top-1 y Top-K:** Mide el porcentaje de consultas donde el documento relevante apareció en los primeros K resultados.

- LLM Judge Score: Evaluación cualitativa usando Ollama como juez para determinar si el contexto recuperado realmente responde a la consulta.

3. Plataforma Web Operativa

- Interfaz Next.js 15: Aplicación web moderna con autenticación RBAC mediante Clerk para gestión de roles y accesos.
- API Python con FastAPI/UVicorn: Backend que ejecuta el agente RAG con capacidad de streaming de respuestas.
- Integración Cloud: Despliegue en Vercel con tunelización mediante Ngrok, permitiendo comunicación entre frontend y LLM local.
- Gestión de Modelos: Soporte dual para modelos locales (Ollama con Phi-3/Llama3) y cloud (Google Gemini).

4. Sistema de Configuración Dinámica

- Manifiestos por cliente: Cada cliente tiene un settings.json y rag.json que define su personalidad, rol del asistente, reglas de oro, y estilo de respuesta.
- Ingesta modular: Sistema de carga que soporta PDFs, archivos web (HTML/JSON/TXT) y tablas estructuradas (CSV/MD).
- Telemetría y Logging: Sistema integrado de traza de razonamiento que muestra paso a paso cómo el agente procesa cada consulta.

5. Pruebas y Validación

- Notebooks de evaluación: Se crearon notebooks (eval_banco.ipynb, analisis_embeddings_completo.ipynb) que demuestran:
 - Carga exitosa de 18 documentos sobre productos de seguros (SVR, SHG, SPP).
 - Generación y indexación de 44 chunks con metadatos asociados.
 - Capacidad de búsqueda semántica con scores de similitud controlados.

6. Métricas de Rendimiento

Los resultados de evaluación incluyen:

- Tiempo de chunking: 0.0001 - 0.005 segundos según configuración.
- Tiempo de embedding: 0.2 - 2.0 segundos para conjuntos de chunks (dependiendo del modelo).
- Consumo de memoria: Monitoreado en rango de 200-500 MB durante operación normal.
- Total de tokens por configuración: Rango de 5,000 - 50,000 tokens según tamaño de chunk y overlap.

3. Discusión

Contribución Principal

Este trabajo demuestra una implementación viable y escalable de RAG empresarial que resuelve el problema fundamental de preservación y democratización del conocimiento técnico dentro de organizaciones. Las principales contribuciones son:

1. Arquitectura Modular y Extensible: El patrón de herencia (BaseModel → ClienteEspecífico) permite agregar nuevos clientes sin tocar el código core, validando el principio DRY (Don't Repeat Yourself).
2. Control Anti-Alucinación Efectivo: Mediante el filtrado de similitud semántica ($\text{score} < 0.85$) y el protocolo [USAR_TABLA], se reduce significativamente el riesgo de que la IA genere información no fundamentada en los documentos reales.
3. Evaluación Multi-Métrica Rigurosa: El framework de evaluación (MRR, Accuracy, LLM Judge) proporciona una base objetiva para elegir el modelo de embedding más apropiado según el caso de uso, no solo por velocidad sino por calidad semántica.
4. Integración Full-Stack: Desde el backend Python con LangChain hasta el frontend Next.js con Clerk, demostrando que sistemas RAG complejos pueden ser realizados con herramientas modernas sin depender de soluciones propietarias monolíticas.

Hallazgos Clave

- Los modelos multilingües funcionan mejor en español: Los modelos específicos para múltiples idiomas (e5-base, distiluse-multilingual) mostraron mejor performance que alternativas monolingües.
- El trade-off chunk size vs. overlap es crítico: Chunks muy grandes pierden granularidad; muy pequeños generan fragmentación. El rango óptimo identificado fue 800-1200 caracteres con overlap de 150-200.

- Ollama local es viable para evaluación: Aunque más lento que APIs cloud, permite un ciclo de desarrollo iterativo sin depender de cuotas externas (como se evidenció en los errores de rate-limiting de Gemini).

Limitaciones Identificadas

1. Dependencia de Calidad Documental: El sistema es tan bueno como los documentos ingresados. PDFs con mala OCR o textos desorganizados reducen la efectividad del RAG.
2. Contexto de Ventana: El LLM tiene un límite de tokens de contexto. Consultas que requieren combinar múltiples documentos pueden fragmentarse.
3. Evaluación en Producción: El LLMJudge es una heurística, no una métrica definitiva. Se recomienda evaluación humana periódica.
4. Escalabilidad de Supabase: Aunque el vector store en Supabase funciona, para millones de vectores podría requerirse una solución especializada (Pinecone, Weaviate, Qdrant).

Líneas Futuras de Trabajo

4. Fine-tuning de Embeddings: Entrenar un modelo de embedding específico para el dominio de seguros usando técnicas de contrastive learning (Sentence-Transformers con corpus de pares Q-A etiquetados).
5. Implementación de Query Expansion: Técnicas como HyDE (Hypothetical Document Embeddings) para mejorar retrieval cuando la consulta inicial es vaga o muy corta.
6. Multi-hop Reasoning: Extender el sistema para soportar consultas que requieren razonamiento sobre múltiples documentos en cadena (ej: "¿Qué beneficios tiene el SVR y cómo se emite?").
7. Feedback Loop de Usuarios: Sistema de thumbs-up/down que alimente un modelo de reranking para que el sistema aprenda qué documentos son realmente útiles según la experiencia real.
8. Integración de Extracted Tables: Actualmente los archivos tipo "tabla" no se embedden. Implementar OCR + parsing de tablas para hacer searchable la información estructurada.

9. Knowledge Graphs: Construir un grafo de entidades (productos, procesos, actores) para soportar consultas más complejas tipo "¿Qué procesos afectan el producto SVR?".
10. Multimodal RAG: Agregar soporte para imágenes y diagramas (usando modelos como LLaVA o GPT-4V) para consultas sobre diagramas de flujo o gráficos.
11. A/B Testing Framework: Sistema de experimentación que permita comparar diferentes configuraciones (modelos, chunk sizes, prompts) contra métricas de éxito usuario-céntrico en tiempo real.
12. Explicabilidad Mejorada: Mostrar al usuario no solo la respuesta sino el fragmento específico del documento (con cita) de donde proviene cada afirmación.
13. Caching Inteligente: Memorizar consultas y respuestas frecuentes para reducir latencia y costo computacional en preguntas repetitivas (comunes en soportes técnicos).

Conclusión

Este Trabajo de Fin de Máster ha demostrado de forma integral que la arquitectura RAG es un patrón viable, escalable y necesario para organizaciones que requieren sistemas de IA confiables y auditables sin necesidad de reentrenamiento de modelos base. A través del desarrollo de una plataforma completa que integra componentes de recuperación semántica, gestión de conocimiento multi-tenant, evaluación rigurosa y una interfaz moderna, se ha validado un ciclo de vida completo desde la ingesta de documentos hasta la respuesta fundamentada en contexto.

Los resultados empíricos demuestran que: (1) la implementación modular de handlers (RAG, tablas, búsqueda directa) permite enrutar automáticamente consultas hacia el mecanismo más apropiado, mejorando tanto precisión como eficiencia; (2) el control anti-alucinación mediante filtrado de similitud semántica ($\text{score} < 0.85$) y protocolos explícitos ([USAR_TABLA]) reduce significativamente el riesgo de generación no fundamentada de información; (3) la evaluación multi-métrica (MRR, Accuracy Top-K, LLM Judge) proporciona una base objetiva para la selección de componentes, permitiendo benchmark riguroso entre 7 modelos de embedding bajo 9 configuraciones

de chunking diferentes; (4) la integración full-stack (Python/LangChain en backend, Next.js 15 en frontend, Supabase para persistencia, Clerk para RBAC) demuestra viabilidad técnica sin dependencias de soluciones propietarias monolíticas.

Operacionalmente, el sistema ha indexado exitosamente 44 fragmentos de documentación técnica con metadatos asociados, implementado streaming de respuestas con trazabilidad de razonamiento paso-a-paso, y desplegado en producción con tunelización segura (Ngrok). La arquitectura de herencia (BaseModel → ClienteEspecífico) valida el principio DRY, permitiendo agregar nuevos clientes/dominios sin tocar código core. Sin embargo, subsisten limitaciones inherentes: la calidad del RAG depende críticamente de la calidad documental de entrada; el contexto de ventana impone límites a consultas multi-documento; la evaluación mediante LLMJudge es heurística, no definitiva; y la escalabilidad a millones de vectores podría requerir infraestructura especializada.

Las líneas futuras prioritarias incluyen: fine-tuning de embeddings específicos al dominio mediante contrastive learning; implementación de query expansion (HyDE); multi-hop reasoning para consultas complejas; feedback loops de usuario para reranking adaptativo; OCR + parsing de tablas para información estructurada; knowledge graphs para razonamiento sobre entidades; extensión multimodal (imágenes/diagramas); frameworks de A/B testing usuario-céntrico; y mecanismos de explicabilidad mejorada con citas verificables.

En conclusión, este trabajo establece un precedente de implementación rigurosa de RAG empresarial, demostrando que sistemas de IA confiables no requieren modelos gigantes ni reentrenamiento, sino arquitectura sólida, evaluación sistemática y feedback continuo. Los cimientos técnicos y metodológicos aquí sentados posibilitan que cualquier organización con documentación técnica despliegue en semanas un asistente inteligente auditable, escalable y mantenible—transformando el conocimiento organizacional de activo pasivo en herramienta competitiva activa.

14. Impacto Social

En esta sección se ha de vincular el proyecto con al menos dos Objetivos de desarrollo Sostenible (ODS) que han de aparecer también en la portada del TFM. Además, ha de recoger las consideraciones éticas o legales en torno a privacidad de datos que se consideren relevantes.

Las imágenes de portada se pueden extraer de aquí:

<https://www.un.org/sustainabledevelopment/es/news/communications-material/>

15. Bibliografia

Côrte-Real, N., Oliveira, T. & Ruivo, P. (2017). Assessing business value of Big Data Analytics in European firms. *Journal of Business Research*, 70, 379-390.

Mikalef et al., 2020; Côrte-Real et al., 2016).

16. Apéndices

10.1. Entrevistas

Bloque 1.1 – Relevancia de los datos y del conocimiento en las organizaciones actuales

Objetivo del bloque

Evidenciar que los datos y el conocimiento son activos críticos, independientemente del tamaño o sector de la organización.

Preguntas sugeridas (comunes a banco y estudio contable)

1. ¿Qué papel juegan actualmente los datos y la información interna en el desarrollo de las tareas diarias de su organización?
2. ¿Considera que el conocimiento sobre procesos, sistemas o normativa es un activo clave para el funcionamiento del negocio?
¿Por qué?
3. En su experiencia, ¿qué impacto tiene contar (o no) con información clara y accesible a la hora de tomar decisiones operativas o de negocio?
4. ¿Diría que la dependencia del conocimiento interno ha aumentado en los últimos años? ¿A qué lo atribuye?

Bloque 1.2 – Consecuencias de una gestión inadecuada de los datos y del conocimiento

Objetivo del bloque

Mostrar efectos concretos de la falta de documentación, gobernanza y accesibilidad.

Preguntas sugeridas

5. ¿Existen en su organización dificultades para acceder a la información necesaria cuando surge una necesidad concreta?
6. ¿Con qué frecuencia se encuentran con datos, tablas, documentos o procesos que no están suficientemente documentados o cuya interpretación no es clara?
7. ¿Qué consecuencias prácticas genera esta falta de documentación o claridad?
(por ejemplo: pérdida de tiempo, errores, dependencia de otras personas, retrabajo)
8. ¿Ha ocurrido que se tomen decisiones o se desarrollen soluciones basadas en interpretaciones incorrectas de los datos?
9. ¿Considera que la falta de una gobernanza clara de los datos afecta al rendimiento o eficiencia del modelo de negocio?

Bloque 1.3 – Complejidad y heterogeneidad según el tipo de organización

Objetivo del bloque

Comparar banco vs pyme, mostrando diferencias de escala pero similitudes estructurales.

Preguntas para institución financiera (Banco)

10. ¿Cómo describiría el volumen y la complejidad de los datos con los que trabajan habitualmente?
11. ¿Intervienen múltiples sistemas, bases de datos o equipos en el acceso y uso de la información?

12. ¿La especialización de roles hace que el conocimiento esté distribuido o concentrado en determinadas personas o áreas?

Preguntas para estudio contable (Pyme)

13. ¿Cómo se gestiona el conocimiento clave en el día a día del estudio?
14. ¿Existe concentración del conocimiento en pocas personas? ¿Cómo impacta esto en la capacitación de nuevos empleados?
15. ¿La actualización normativa y la información fiscal representa una dificultad relevante para el trabajo cotidiano?

Preguntas comunes (comparativas)

16. Aunque el volumen de datos sea distinto, ¿considera que los problemas de acceso, documentación y dependencia del conocimiento son similares a los de otras organizaciones?

Bloque 1.4 – Identificación y agrupación de las principales urgencias detectadas

Objetivo del bloque

Validar empíricamente las “urgencias” y justificar su agrupación.

Preguntas sugeridas (comunes)

Cuando surge una duda o necesidad concreta, ¿a quién suele recurrirse para resolverla?

17. ¿Con qué frecuencia la resolución de una tarea depende del conocimiento de una persona específica?
18. ¿Qué tipo de tareas considera más costosas en tiempo debido a la falta de información clara o accesible?

19. ¿Se repiten consultas, cruces de información o tareas que ya han sido resueltas anteriormente?
20. ¿Existen directrices o demandas desde niveles directivos para mejorar la autonomía, eficiencia o rapidez en el trabajo con datos?
21. ¿Cómo se gestionan actualmente las solicitudes a equipos de IT o soporte?
- ¿Considera que este proceso podría ser más eficiente?

a. Apéndice 1: Dashboard

[Incluir enlace a dashboard, si aplica]

b. Apéndice 2: Código y datos

[Incluir enlace a repositorio con código y datos, si aplica]

